Rapport Technique Frontend

Application Mobile Flutter - Bank Assistant

Équipe de Développement:
Osama Mansouri
Sohaib Laarichi
Salah Eddine Aboulkameh
Yassir Rabai

Encadrant:
Hanae Sbai

Date de création: 22 décembre 2025

Version: 1.0

22 décembre 2025

Rapport Technique Frontend

Application Mobile Flutter

Bank Assistant - Interface Utilisateur

Version 1.0

22 décembre 2025

Table des matières

Résumé Exécutif **7**

1 Introduction **8**
1.1 Contexte du Projet 8
1.2 Objectifs du Frontend 8
1.3 Portée du Document 8
1.4 Équipe et Responsabilités 9

2 Architecture Globale de l'Application **10**
2.1 Vue d'Ensemble de l'Architecture 10
2.1.1 Structure en Couches 12
2.2 Choix Technologiques 13
2.2.1 Flutter Framework 13
2.2.2 Dart Langage de Programmation 14
2.2.3 Provider pour la Gestion d'État 14
2.2.4 Packages Principaux Utilisés 14
2.3 Structure du Projet 15
2.3.1 Organisation des Répertoires 15
2.3.2 Conventions de Nommage 16

3 Modèles de Données **17**
3.1 Architecture des Modèles 17
3.1.1 Modèle BankAccount 17
3.1.2 Modèle Transaction 18
3.1.3 Modèle Customer 19
3.1.4 Génération Automatique du Code 20

4 Services et Communication API **21**
4.1 Architecture des Services 21
4.1.1 Service API Principal 22
4.1.2 Service d'Authentification 23
4.1.3 Service de Stockage Local 24
4.2 Gestion des Erreurs 25

5 Gestion d'État avec Provider **26**
5.1 Architecture Provider 26
5.1.1 Provider d'Authentification 26

5.1.2 Provider de Comptes . . . 28
5.2 Pattern Observer et Réactivité . . . 29

6 Écrans et Navigation **30**
6.1 Système de Navigation . . . 30
6.1.1 Configuration des Routes . . . 30
6.2 Écrans Principaux . . . 31
6.2.1 Écran de Connexion . . . 31
6.2.2 Dashboard Client . . . 32
6.2.3 Dashboard Administrateur . . . 33
6.2.4 Écran de Gestion des Comptes . . . 35
6.2.5 Écran de Détails de Compte . . . 36
6.2.6 Écran de Gestion des Clients . . . 37
6.2.7 Écran de Gestion des Transactions . . . 39
6.2.8 Écran de Chat . . . 41

7 Widgets Réutilisables et Composants UI **43**
7.1 Design System . . . 43
7.1.1 Thème de l'Application . . . 43
7.1.2 Widget ModernCard . . . 44
7.1.3 Widget DetailCard . . . 45
7.1.4 Widget ModernModal . . . 46

8 Expérience Utilisateur et Interface **49**
8.1 Design Moderne et Professionnel . . . 49
8.1.1 Hierarchie Visuelle . . . 49
8.1.2 Feedback Utilisateur . . . 49
8.2 Responsivité et Adaptabilité . . . 50
8.3 Gestion des États de Chargement . . . 52

9 Sécurité et Authentification **53**
9.1 Gestion de l'Authentification . . . 53
9.2 Contrôle d'Accès . . . 53

10 Performance et Optimisations **54**
10.1 Optimisations de Performance . . . 54
10.2 Gestion de la Mémoire . . . 54

11 Gestion des Erreurs et Robustesse **55**
11.1 Stratégie de Gestion des Erreurs . . . 55
11.2 Logging et Debugging . . . 55

12 Conclusion **56**
12.1 Résumé des Réalisations . . . 56
12.2 Points Forts de l'Application . . . 56
12.3 Perspectives d'Amélioration . . . 57
12.4 Engagement de l'Équipe . . . 57

A Annexe A : Liste des Captures d'Écran **59**

B Annexe B : Structure Complète du Projet **61**

C Annexe C : Liste des Packages et Versions **64**

D Annexe D : Glossaire **65**

Table des figures

2.1 Architecture Globale de l'Application Flutter - Vue C4 11
2.2 Architecture en Couches de l'Application Flutter 12
2.3 Diagramme des Composants et Flux de Données 13
2.4 Structure Hiérarchique du Projet Flutter 15

4.1 Architecture des Services et Communication avec l'API 21

5.1 Pattern Provider - Gestion d'État Réactive 26

6.1 Écran de connexion - Interface moderne avec validation en temps réel . . . 32
6.2 Dashboard client - Vue personnalisée pour les clients 33
6.3 Dashboard administrateur - Vue d'ensemble complète du système bancaire 34
6.4 Dashboard administrateur - Section statistiques et indicateurs de performance 35
6.5 Écran de gestion des comptes - Liste des comptes avec filtres et recherche . 36
6.6 Écran de détails de compte - Informations complètes et historique des transactions 37
6.7 Écran de gestion des clients - Liste complète avec recherche et filtres 38
6.8 Écran de détails de client - Informations complètes du client 39
6.9 Écran de gestion des transactions - Liste avec filtres et indicateurs visuels . 40
6.10 Écran de détails de transaction - Informations complètes de la transaction 41
6.11 Écran de chat - Interface de messagerie avec l'assistant virtuel intelligent . 42

7.1 Exemple d'utilisation du widget ModernCard - Cartes avec design moderne 44
7.2 Exemple d'utilisation du widget ModernModal - Modal moderne pour formulaires 47

8.1 Vue d'ensemble du design - Interface moderne et professionnelle 49
8.2 Responsivité de l'application - Adaptation aux différentes tailles d'écran . . 51

Liste des tableaux

A.1 Liste des Captures d'Écran Requises 60

C.1 Packages Principaux Utilisés 64

Résumé Exécutif

Ce document présente une analyse complète et détaillée de l'application frontend développée avec Flutter pour le système Bank Assistant. L'application mobile offre une interface utilisateur moderne, intuitive et performante permettant aux administrateurs et aux clients de gérer efficacement leurs opérations bancaires.

Le présent rapport décrit en profondeur l'architecture frontend, les choix technologiques, les composants développés, les patterns de conception utilisés, ainsi que les fonctionnalités implémentées. Il couvre également les aspects de performance, d'expérience utilisateur, de gestion d'état, de navigation, et de sécurité côté client.

L'application Flutter a été conçue pour offrir une expérience utilisateur exceptionnelle sur toutes les plateformes (iOS, Android, Web) avec un code unique, tout en respectant les meilleures pratiques de développement mobile moderne.

Introduction

1.1 Contexte du Projet

Le système Bank Assistant nécessite une interface utilisateur moderne et accessible permettant aux différents acteurs (administrateurs et clients) d'interagir avec les services bancaires. L'application frontend Flutter a été développée pour répondre à ce besoin en offrant une expérience utilisateur fluide et intuitive sur toutes les plateformes.

L'application mobile permet aux utilisateurs de consulter leurs comptes, effectuer des transactions, gérer leurs profils, interagir avec un assistant virtuel intelligent, et pour les administrateurs, de gérer l'ensemble du système bancaire via un tableau de bord complet.

1.2 Objectifs du Frontend

Les objectifs principaux de l'application frontend sont les suivants :

1. Fournir une interface utilisateur moderne et professionnelle adaptée au secteur bancaire
2. Assurer une expérience utilisateur fluide et intuitive sur toutes les plateformes
3. Implémenter une architecture scalable et maintenable
4. Garantir la sécurité des données utilisateur côté client
5. Optimiser les performances pour une utilisation fluide même sur des appareils moins puissants
6. Assurer l'accessibilité et la responsivité sur différentes tailles d'écran
7. Implémenter une gestion d'état robuste et prévisible

1.3 Portée du Document

Ce document couvre l'ensemble des aspects techniques du frontend Flutter :

— L'architecture globale de l'application
— Les choix technologiques et leurs justifications
— La structure du projet et l'organisation du code
— Les modèles de données et leur sérialisation

— Les services et la communication avec l'API backend
— La gestion d'état avec Provider
— Les écrans et leurs fonctionnalités
— Les widgets réutilisables et les composants UI
— Le système de navigation et de routage
— Le thème et le design system
— La gestion de l'authentification et de la sécurité
— Les optimisations de performance
— La gestion des erreurs et des états de chargement

1.4 Équipe et Responsabilités

Équipe de Développement Frontend :

— **Osama Mansouri** : Architecte frontend et responsable de l'architecture globale
— **Sohaib Laarichi** : Développeur Flutter et responsable des composants UI
— **Salah Eddine Aboulkameh** : Développeur full-stack et responsable de l'intégration API
— **Yassir Rabai** : DevOps et responsable de la configuration de build et déploiement

Encadrant : Hanae Sbai

Architecture Globale de l'Application

2.1 Vue d'Ensemble de l'Architecture

L'application Flutter suit une architecture en couches bien définie, inspirée des meilleures pratiques de développement mobile. Cette architecture garantit la séparation des responsabilités, la testabilité, la maintenabilité et la scalabilité du code.

L'architecture est organisée selon le principe de séparation des préoccupations (Separation of Concerns), où chaque couche a une responsabilité claire et bien définie. Cette approche facilite la compréhension du code, les tests unitaires, et les modifications futures.

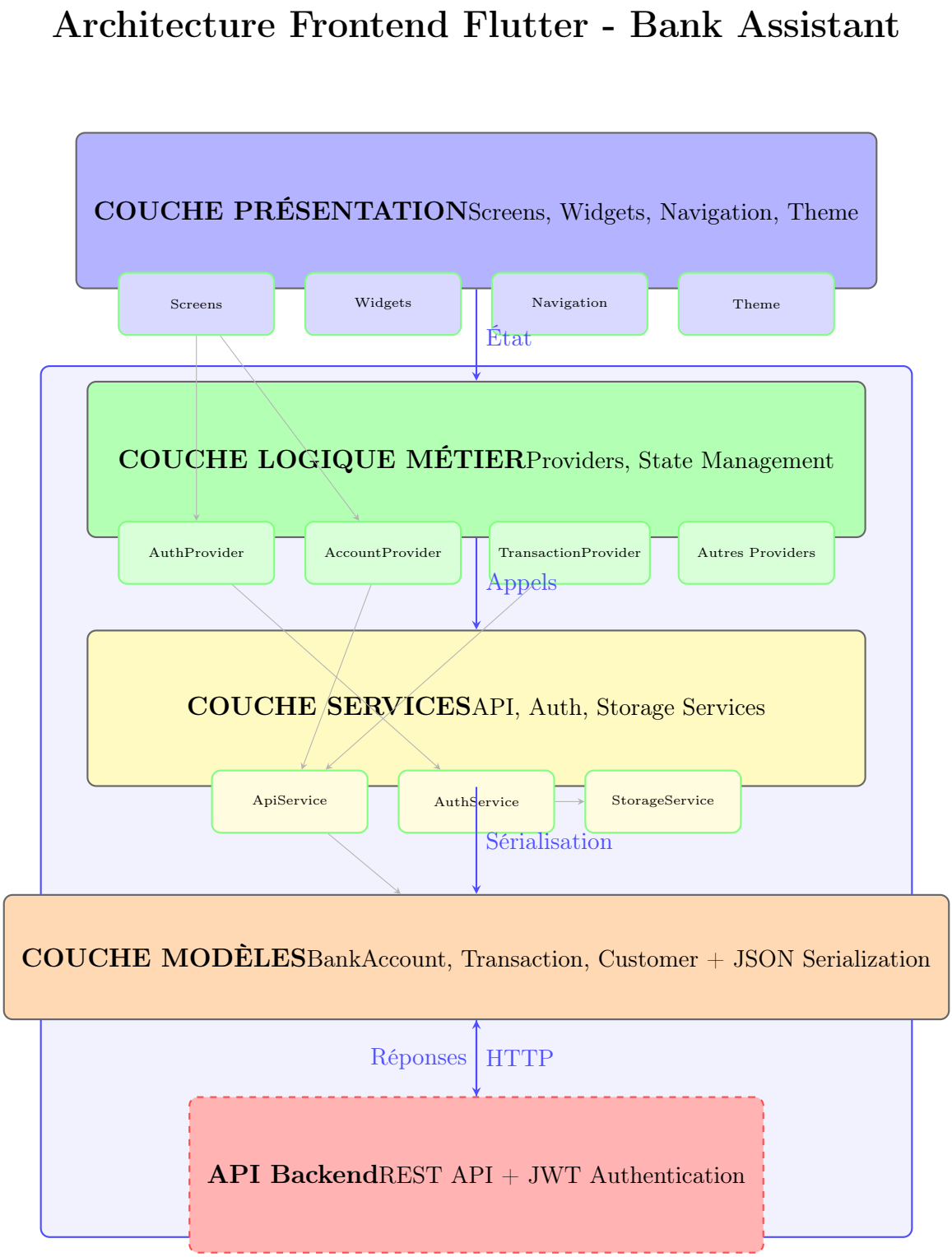


FIGURE 2.1 – Architecture Globale de l'Application Flutter - Vue C4

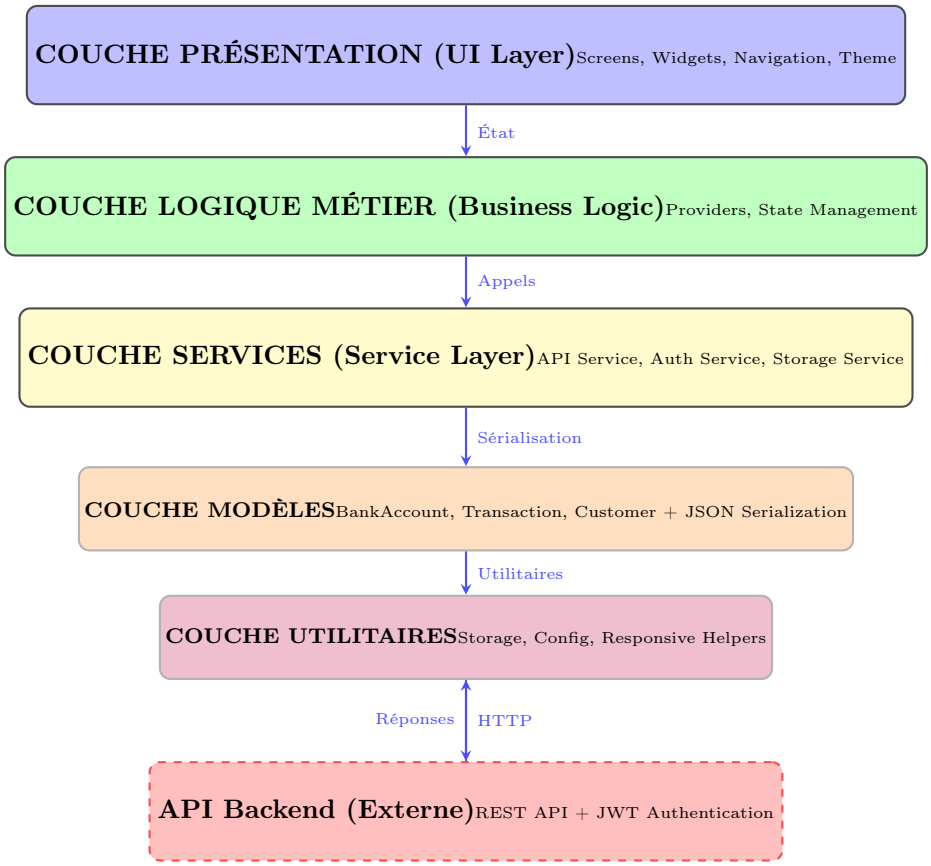


FIGURE 2.2 – Architecture en Couches de l'Application Flutter

2.1.1 Structure en Couches

L'application est structurée en plusieurs couches distinctes, comme illustré dans la figure 2.2 :

Couche Présentation (UI Layer) : Cette couche contient tous les éléments visuels de l'application. Elle est composée des écrans (screens), des widgets réutilisables, et des composants d'interface utilisateur. Cette couche est responsable de l'affichage des données et de la capture des interactions utilisateur.

Couche Logique Métier (Business Logic Layer) : Cette couche contient les providers qui gèrent l'état de l'application et la logique métier. Les providers utilisent le pattern Provider de Flutter pour gérer l'état de manière réactive et prévisible. Cette couche orchestre les appels aux services et transforme les données pour la présentation.

Couche Services (Service Layer) : Cette couche contient tous les services qui communiquent avec l'API backend. Elle encapsule la logique de communication réseau, la gestion des erreurs réseau, et la transformation des données. Les services sont indépendants de l'interface utilisateur et peuvent être facilement testés.

Couche Modèles (Model Layer) : Cette couche contient les modèles de données qui représentent les entités métier. Les modèles sont générés automatiquement à partir des définitions JSON pour garantir la cohérence avec l'API backend. Cette couche assure la sérialisation et la désérialisation des données.

Couche Utilitaires (Utility Layer) : Cette couche contient les utilitaires, les hel-

pers, et les fonctions utilitaires utilisées dans toute l'application. Elle inclut la gestion du stockage local, la configuration de l'application, les fonctions de formatage, et les outils de responsivité.

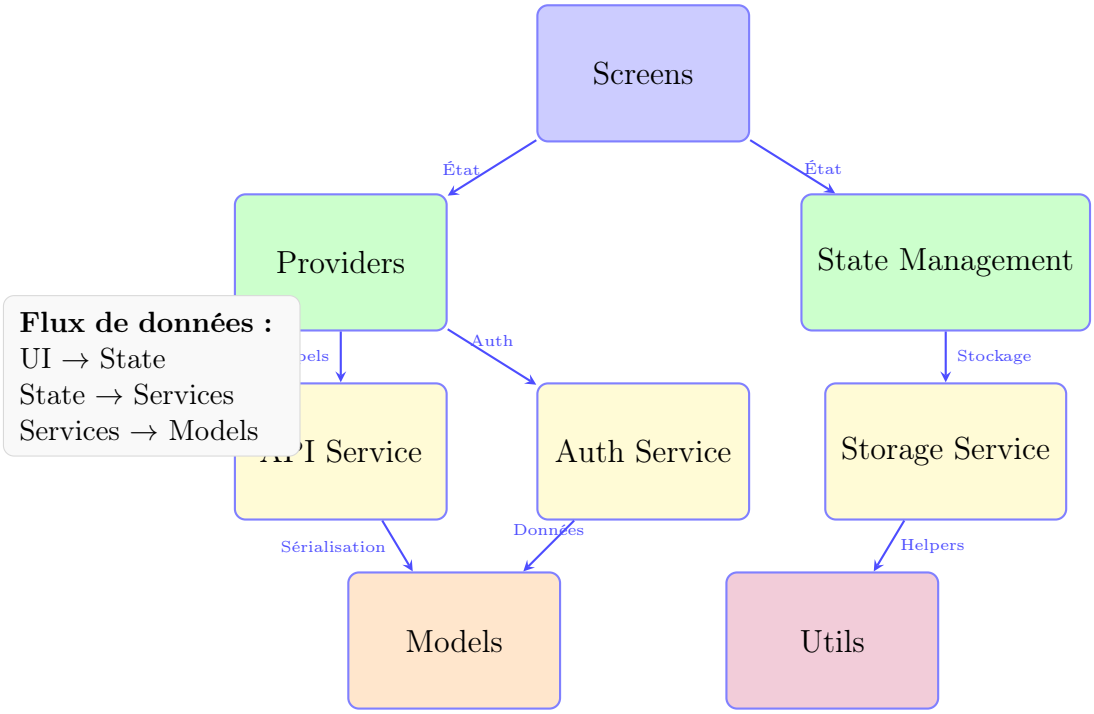


FIGURE 2.3 – Diagramme des Composants et Flux de Données

2.2 Choix Technologiques

2.2.1 Flutter Framework

Flutter a été choisi comme framework principal pour le développement de l'application mobile pour plusieurs raisons stratégiques :

Multi-plateforme Native : Flutter permet de développer une seule application qui fonctionne nativement sur iOS, Android, et Web. Cela réduit considérablement le temps de développement et les coûts de maintenance, tout en offrant des performances natives grâce à la compilation en code machine.

Performance Exceptionnelle : Flutter utilise le moteur de rendu Skia et compile en code natif, ce qui garantit des performances élevées et une fluidité d'animation à 60 fps (ou même 120 fps sur les appareils récents). Cette performance est cruciale pour une application bancaire où la réactivité est essentielle.

UI Moderne et Personnalisable : Flutter offre un système de widgets riche et hautement personnalisable. Le Material Design 3 permet de créer des interfaces modernes et professionnelles adaptées au secteur bancaire, avec des animations fluides et des transitions élégantes.

Hot Reload et Productivité : Le système de Hot Reload de Flutter permet de voir les modifications en temps réel sans redémarrer l'application, ce qui accélère considérablement le développement et le débogage.

Écosystème Riche : Flutter dispose d'un écosystème de packages riche et bien maintenu, avec des solutions pour presque tous les besoins : gestion d'état, navigation, authentification, stockage local, etc.

2.2.2 Dart Langage de Programmation

Dart est le langage de programmation utilisé par Flutter. Il a été choisi pour plusieurs avantages :

Syntaxe Moderne : Dart offre une syntaxe moderne et expressive, similaire à Java et C#, ce qui facilite l'apprentissage pour les développeurs ayant une expérience avec ces langages.

Typage Fort : Dart est un langage typé statiquement avec inférence de type, ce qui permet de détecter les erreurs à la compilation et améliore la productivité grâce à l'autocomplétion dans les IDE.

Asynchrone et Réactif : Dart dispose d'excellents outils pour la programmation asynchrone avec les mots-clés `async` et `await`, ce qui simplifie grandement la gestion des opérations réseau et des appels API.

Null Safety : Dart dispose d'un système de null safety intégré qui prévient les erreurs de null pointer exception, améliorant ainsi la robustesse du code.

2.2.3 Provider pour la Gestion d'État

Provider a été choisi comme solution de gestion d'état pour l'application. Ce choix est justifié par plusieurs raisons :

Simplicité et Clarté : Provider est simple à comprendre et à utiliser, ce qui facilite la maintenance du code et l'onboarding de nouveaux développeurs. Il suit le pattern Observer et s'intègre naturellement avec Flutter.

Performance Optimisée : Provider optimise automatiquement les reconstructions de widgets en ne reconstruisant que les widgets qui dépendent des données modifiées. Cela garantit des performances optimales même avec des applications complexes.

Testabilité : Provider facilite grandement les tests unitaires et d'intégration en permettant de mocker facilement les providers et d'isoler la logique métier.

Écosystème Flutter : Provider est développé par l'équipe Flutter et est recommandé comme solution officielle pour la gestion d'état, garantissant une compatibilité et une maintenance à long terme.

2.2.4 Packages Principaux Utilisés

L'application utilise plusieurs packages essentiels pour son fonctionnement :

http : Package officiel pour effectuer des requêtes HTTP vers l'API backend. Il offre une API simple et intuitive pour les appels GET, POST, PUT, DELETE, avec support des headers personnalisés et de la gestion des erreurs.

json_serializable : Package pour la génération automatique du code de sérialisation JSON. Il génère les méthodes `toJson()` et `fromJson()` à partir d'annotations, garantissant la cohérence avec l'API backend et réduisant les erreurs de typage.

shared_preferences : Package pour le stockage local de données simples (clés-valeurs). Il est utilisé pour stocker les tokens d'authentification, les préférences utilisateur, et les données de session.

intl : Package pour l'internationalisation et le formatage des dates, nombres, et devises. Il permet de formater les montants monétaires selon les standards bancaires et de gérer les différentes locales.

flutter_dotenv : Package pour charger les variables d'environnement depuis un fichier `.env`. Il permet de configurer l'URL de l'API, les clés API, et autres configurations sans modifier le code.

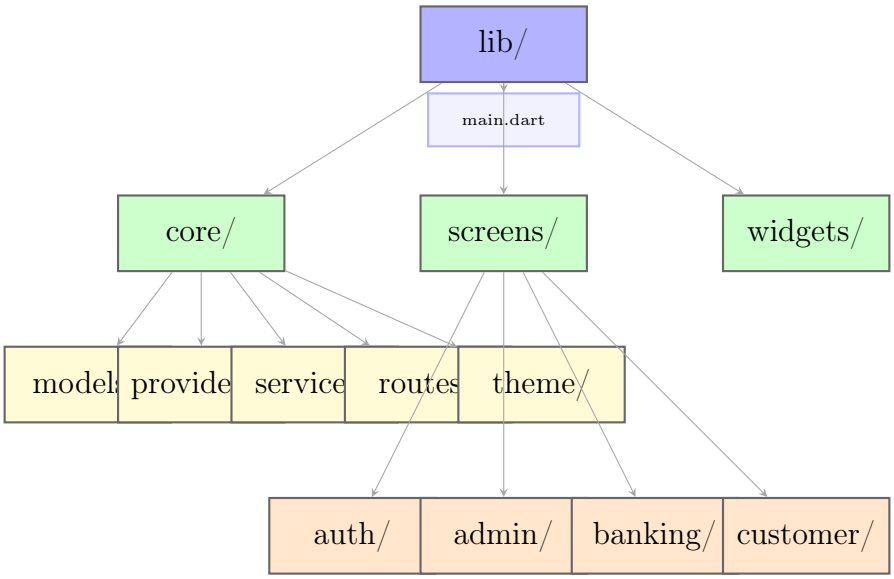


FIGURE 2.4 – Structure Hiérarchique du Projet Flutter

2.3 Structure du Projet

2.3.1 Organisation des Répertoires

Le projet Flutter est organisé de manière modulaire et hiérarchique pour faciliter la navigation et la maintenance :

Répertoire `lib/` : Contient tout le code source de l'application Dart. C'est le répertoire principal où réside toute la logique de l'application.

Répertoire `lib/core/` : Contient le code fondamental et réutilisable de l'application. Ce répertoire inclut :

- `config/` : Configuration de l'application (URLs API, constantes, etc.)
- `models/` : Modèles de données et leurs fichiers de sérialisation générés
- `providers/` : Providers pour la gestion d'état
- `services/` : Services pour la communication avec l'API et autres services externes
- `routes/` : Configuration du système de routage
- `theme/` : Thème et styles de l'application
- `utils/` : Fonctions utilitaires et helpers

Répertoire `lib/screens/` : Contient tous les écrans de l'application, organisés par domaine fonctionnel :

- `auth/` : Écrans d'authentification (login, profile)
- `admin/` : Écrans réservés aux administrateurs (dashboard admin, knowledge base)
- `banking/` : Écrans de gestion bancaire (comptes, transactions, clients, produits)
- `customer/` : Écrans pour les clients (dashboard client, mes comptes, mes transactions)
- `leads/` : Écrans de gestion des leads commerciaux
- `chat/` : Écran de chat avec l'assistant virtuel
- `settings/` : Écran de paramètres

Répertoire `lib/widgets/` : Contient tous les widgets réutilisables utilisés dans plusieurs écrans. Ces widgets encapsulent des fonctionnalités UI communes et peuvent être réutilisés dans différents contextes.

Répertoire `lib/main.dart` : Point d'entrée principal de l'application. Ce fichier initialise l'application, configure les providers, définit le thème, et lance l'écran initial.

2.3.2 Conventions de Nommage

L'application suit des conventions de nommage strictes pour garantir la cohérence et la lisibilité du code :

Fichiers : Les fichiers utilisent le snake_case (ex : `account_provider.dart`, `login_screen.dart`).
Classes : Les classes utilisent le PascalCase (ex : `AccountProvider`, `LoginScreen`).
Variables et Méthodes : Utilisent le camelCase (ex : `loadAccounts()`, `isLoading`).
Constantes : Utilisent le UPPER_SNAKE_CASE (ex : `API_BASE_URL`, `MAX_RETRY_ATTEMPTS`).
Widgets Privés : Les widgets privés commencent par un underscore (ex : `_buildAccountCard()`).

Modèles de Données

3.1 Architecture des Modèles

Les modèles de données dans l'application Flutter représentent les entités métier du système bancaire. Ces modèles sont générés automatiquement à partir des définitions JSON pour garantir la cohérence avec l'API backend et éviter les erreurs de typage.

Chaque modèle est composé de deux fichiers : un fichier source (`.dart`) contenant la définition de la classe avec les annotations, et un fichier généré (`.g.dart`) contenant le code de sérialisation généré automatiquement.

3.1.1 Modèle BankAccount

Le modèle `BankAccount` représente un compte bancaire dans le système. Il contient toutes les informations nécessaires pour afficher et gérer un compte :

Listing 3.1 – Modèle BankAccount

```
@JsonSerializable()
class BankAccount {
  final int id;
  final String accountNumber;
  final String accountType;
  final double balance;
  final String currency;
  final int customerId;
  final String? customerName;
  final DateTime createdAt;
  final DateTime? updatedAt;
  final String status;

  BankAccount({
    required this.id,
    required this.accountNumber,
    required this.accountType,
    required this.balance,
    required this.currency,
    required this.customerId,
    this.customerName,
```

```
    required this.createdAt,
    this.updatedAt,
    required this.status,
  });

  factory BankAccount.fromJson(Map<String, dynamic> json) =>
      _$BankAccountFromJson(json);

  Map<String, dynamic> toJson() => _$BankAccountToJson(this);
}
```

Ce modèle inclut des informations essentielles comme le numéro de compte, le type de compte (CHECKING, SAVINGS, etc.), le solde, la devise, et l'identifiant du client propriétaire. Le champ `customerName` est optionnel et peut être enrichi par l'API backend pour afficher le nom du client directement dans les listes de comptes.

3.1.2 Modèle Transaction

Le modèle `Transaction` représente une transaction bancaire. Il capture tous les détails d'une opération financière :

Listing 3.2 – Modèle Transaction

```
@JsonSerializable()
class Transaction {
  final int id;
  final String transactionNumber;
  final String transactionType;
  final double amount;
  final String currency;
  final int accountId;
  final String? accountNumber;
  final int? customerId;
  final String? customerName;
  final String? description;
  final DateTime transactionDate;
  final String status;

  Transaction({
    required this.id,
    required this.transactionNumber,
    required this.transactionType,
    required this.amount,
    required this.currency,
    required this.accountId,
    this.accountNumber,
    this.customerId,
    this.customerName,
    this.description,
    required this.transactionDate,
```

```
    required this.status,
  });

  factory Transaction.fromJson(Map<String, dynamic> json) =>
      _$TransactionFromJson(json);

  Map<String, dynamic> toJson() => _$TransactionToJson(this);
}
```

Ce modèle inclut le type de transaction (DEPOSIT, WITHDRAWAL, TRANSFER), le montant, la date, et les informations sur le compte et le client associés. Les champs optionnels comme `customerName` et `accountNumber` permettent d'enrichir l'affichage sans nécessiter des appels API supplémentaires.

3.1.3 Modèle Customer

Le modèle `Customer` représente un client du système bancaire. Il contient les informations personnelles et bancaires du client :

Listing 3.3 – Modèle Customer

```
@JsonSerializable()
class Customer {
  final int id;
  final String customerNumber;
  final String firstName;
  final String lastName;
  final String email;
  final String? phone;
  final DateTime? dateOfBirth;
  final String status;
  final bool kycVerified;

  Customer({
    required this.id,
    required this.customerNumber,
    required this.firstName,
    required this.lastName,
    required this.email,
    this.phone,
    this.dateOfBirth,
    required this.status,
    required this.kycVerified,
  });

  String get fullName => '$firstName $lastName';

  factory Customer.fromJson(Map<String, dynamic> json) =>
      _$CustomerFromJson(json);
```

```
  Map<String, dynamic> toJson() => _$CustomerToJson(this);
}
```

Ce modèle inclut un getter `fullName` qui combine automatiquement le prénom et le nom pour faciliter l'affichage dans l'interface utilisateur.

3.1.4 Génération Automatique du Code

La génération automatique du code de sérialisation est effectuée à l'aide du package `build_runner`. Cette approche présente plusieurs avantages :

Cohérence avec l'API : Le code généré garantit que la structure des modèles correspond exactement à la structure JSON retournée par l'API backend, réduisant ainsi les erreurs de désérialisation.

Maintenance Simplifiée : Lorsque l'API change, il suffit de mettre à jour le modèle et de régénérer le code, sans avoir à écrire manuellement le code de sérialisation.

Performance : Le code généré est optimisé et performant, évitant les réflexions coûteuses au runtime.

Type Safety : Le typage fort de Dart combiné avec la génération de code garantit que les erreurs de typage sont détectées à la compilation.

Services et Communication API

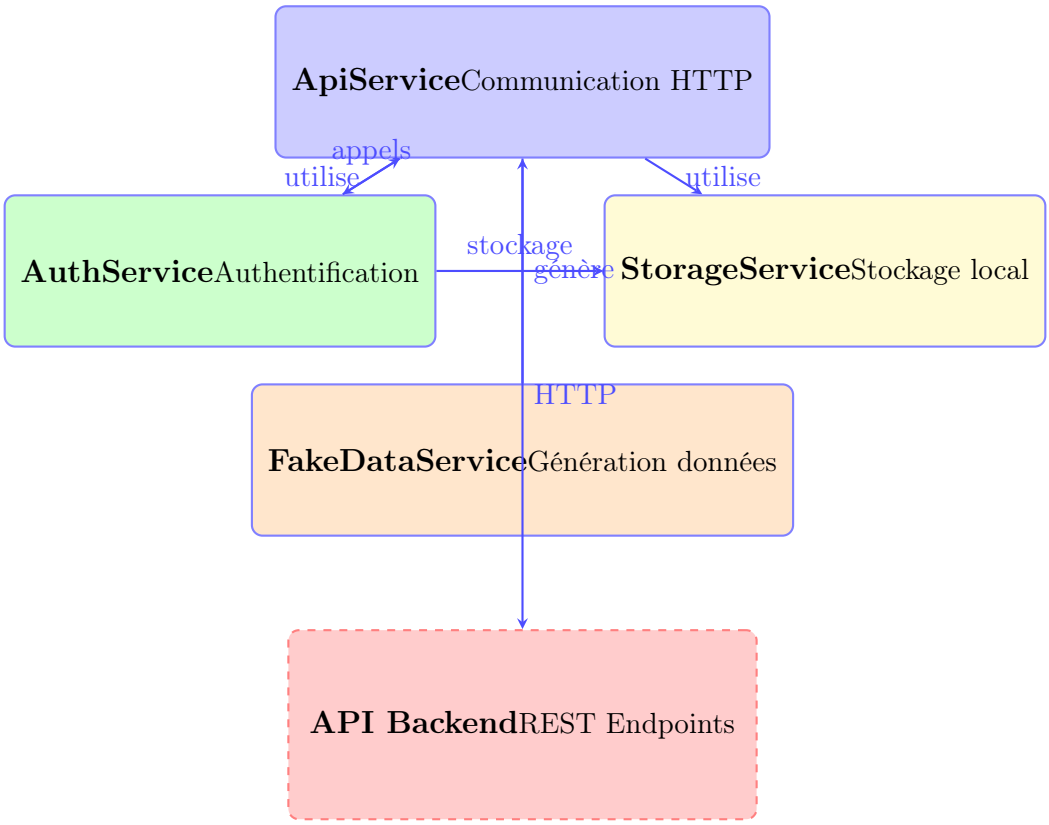


FIGURE 4.1 – Architecture des Services et Communication avec l'API

4.1 Architecture des Services

Les services dans l'application Flutter sont responsables de la communication avec l'API backend. Ils encapsulent toute la logique de communication réseau, la gestion des erreurs, et la transformation des données. La figure 4.1 illustre l'organisation des services et leurs interactions.

4.1.1 Service API Principal

Le `ApiService` est le service central qui gère toutes les communications HTTP avec le backend. Il fournit des méthodes génériques pour effectuer des requêtes GET, POST, PUT, DELETE, avec gestion automatique des headers d'authentification.

Listing 4.1 – Service API Principal

```
class ApiService {
  final String baseUrl;
  final http.Client _client;
  final StorageService _storageService;

  ApiService({
    required this.baseUrl,
    required StorageService storageService,
  }) : _client = http.Client(),
       _storageService = storageService;

  Future<Map<String, String>> _getHeaders() async {
    final token = await _storageService.getToken();
    return {
      'Content-Type': 'application/json',
      'Authorization': token != null ? 'Bearer $token' : '',
    };
  }

  Future<Response> get(String endpoint) async {
    final headers = await _getHeaders();
    final response = await _client.get(
      Uri.parse('$baseUrl$endpoint'),
      headers: headers,
    );
    return _handleResponse(response);
  }

  Future<Response> post(String endpoint, Map<String, dynamic> data)
      async {
    final headers = await _getHeaders();
    final response = await _client.post(
      Uri.parse('$baseUrl$endpoint'),
      headers: headers,
      body: jsonEncode(data),
    );
    return _handleResponse(response);
  }

  Response _handleResponse(http.Response response) {
    if (response.statusCode >= 200 && response.statusCode < 300) {
      return Response.success(response.body);
    } else {
```

```
      return Response.error(
        response.statusCode,
        response.body,
      );
    }
  }
}
```

Ce service gère automatiquement l'ajout du token d'authentification dans les headers de toutes les requêtes, simplifiant ainsi l'utilisation dans les providers et les écrans.

4.1.2 Service d'Authentification

Le `AuthService` gère spécifiquement les opérations d'authentification : connexion, déconnexion, et récupération des informations de l'utilisateur connecté.

Listing 4.2 – Service d'Authentification

```
class AuthService {
  final ApiService _apiService;
  final StorageService _storageService;

  AuthService({
    required ApiService apiService,
    required StorageService storageService,
  }) : _apiService = apiService,
       _storageService = storageService;

  Future<Map<String, dynamic>> login(String username, String
     password) async {
    final response = await _apiService.post('/api/auth/login', {
      'username': username,
      'password': password,
    });

    if (response.isSuccess) {
      final data = jsonDecode(response.data);
      await _storageService.saveToken(data['accessToken']);
      await _storageService.saveRefreshToken(data['refreshToken']);
      return data;
    } else {
      throw Exception(' chec  de la connexion: ${response.error}');
    }
  }

  Future<void> logout() async {
    await _storageService.clearToken();
    await _storageService.clearRefreshToken();
  }
```

```
  Future<Map<String, dynamic>> getCurrentUser() async {
    final response = await _apiService.get('/api/auth/me');
    if (response.isSuccess) {
      return jsonDecode(response.data);
    } else {
      throw Exception('Impossible de r cup rer les informations
         utilisateur');
    }
  }
}
```

Ce service gère le stockage sécurisé des tokens d'authentification et leur récupération pour les requêtes suivantes.

4.1.3 Service de Stockage Local

Le `StorageService` utilise `shared_preferences` pour stocker les données localement sur l'appareil. Il fournit une interface simple et sécurisée pour stocker et récupérer les tokens, les préférences utilisateur, et autres données de session.

Listing 4.3 – Service de Stockage

```
class StorageService {
  final SharedPreferences _prefs;

  StorageService(this._prefs);

  Future<void> saveToken(String token) async {
    await _prefs.setString('access_token', token);
  }

  Future<String?> getToken() async {
    return _prefs.getString('access_token');
  }

  Future<void> clearToken() async {
    await _prefs.remove('access_token');
  }

  Future<void> saveUserInfo(Map<String, dynamic> userInfo) async {
    await _prefs.setString('user_info', jsonEncode(userInfo));
  }

  Future<Map<String, dynamic>?> getUserInfo() async {
    final userInfoStr = _prefs.getString('user_info');
    if (userInfoStr != null) {
      return jsonDecode(userInfoStr) as Map<String, dynamic>;
    }
    return null;
  }
```

```
}
```

Ce service abstrait les détails d'implémentation du stockage local, permettant de changer facilement d'implémentation si nécessaire.

4.2 Gestion des Erreurs

La gestion des erreurs dans l'application est cruciale pour offrir une expérience utilisateur fluide. L'application gère plusieurs types d'erreurs :

Erreurs Réseau : Les erreurs de connexion réseau (timeout, pas de connexion Internet) sont détectées et affichées avec des messages clairs à l'utilisateur.

Erreurs HTTP : Les erreurs HTTP (404, 401, 403, 500, etc.) sont interceptées et transformées en messages utilisateur compréhensibles.

Erreurs de Validation : Les erreurs de validation côté serveur sont affichées dans les formulaires avec des messages spécifiques pour chaque champ.

Erreurs d'Authentification : Les erreurs d'authentification (token expiré, accès non autorisé) déclenchent automatiquement une déconnexion et une redirection vers l'écran de connexion.

Gestion d'État avec Provider

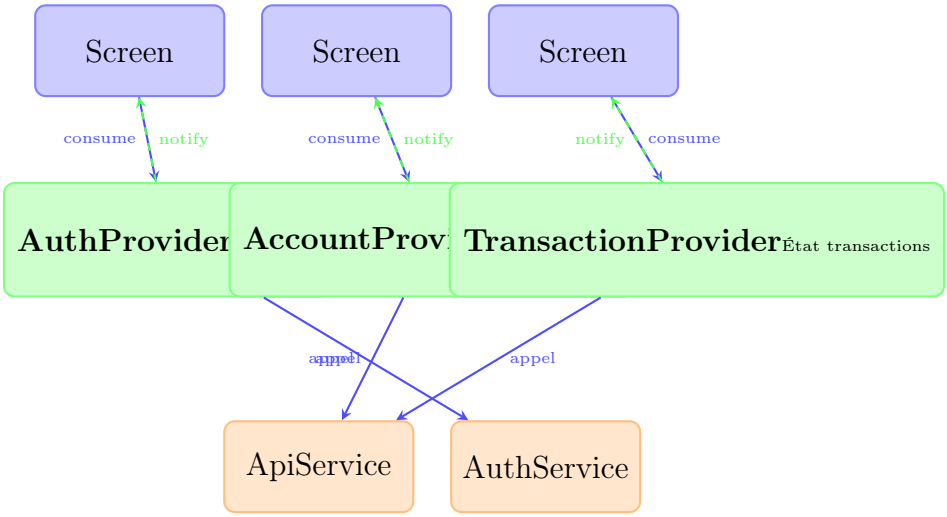


FIGURE 5.1 – Pattern Provider - Gestion d'État Réactive

5.1 Architecture Provider

La gestion d'état dans l'application utilise le pattern Provider, qui permet de gérer l'état de manière réactive et prévisible. Chaque domaine fonctionnel a son propre provider qui encapsule la logique métier et l'état associé. La figure 5.1 illustre le flux de données entre les écrans, les providers et les services.

5.1.1 Provider d'Authentification

Le `AuthProvider` gère l'état d'authentification de l'application. Il maintient l'état de connexion, les informations de l'utilisateur connecté, et expose des méthodes pour se connecter et se déconnecter.

Listing 5.1 – Provider d'Authentification

```
class AuthProvider extends ChangeNotifier {
  final AuthService _authService;
  final StorageService _storageService;
```

```
bool _isAuthenticated = false;
bool _isLoading = false;
Map<String, dynamic>? _currentUser;
String? _error;

AuthProvider({
  required AuthService authService,
  required StorageService storageService,
}) : _authService = authService,
     _storageService = storageService {
  _loadUser();
}

bool get isAuthenticated => _isAuthenticated;
bool get isLoading => _isLoading;
Map<String, dynamic>? get currentUser => _currentUser;
String? get error => _error;

Future<void> login(String username, String password) async {
  _isLoading = true;
  _error = null;
  notifyListeners();

  try {
    final response = await _authService.login(username, password)
       ;
    _currentUser = response;
    _isAuthenticated = true;
    await _storageService.saveUserInfo(response);
  } catch (e) {
    _error = e.toString();
    _isAuthenticated = false;
  } finally {
    _isLoading = false;
    notifyListeners();
  }
}

Future<void> logout() async {
  await _authService.logout();
  _isAuthenticated = false;
  _currentUser = null;
  notifyListeners();
}

Future<void> _loadUser() async {
  final userInfo = await _storageService.getUserInfo();
  if (userInfo != null) {
    _currentUser = userInfo;
```

```
      _isAuthenticated = true;
      notifyListeners();
    }
  }
}
```

Ce provider expose l'état d'authentification de manière réactive, permettant aux écrans de se mettre à jour automatiquement lorsque l'état change.

5.1.2 Provider de Comptes

Le `AccountProvider` gère l'état des comptes bancaires. Il charge les comptes depuis l'API, gère l'état de chargement, et expose les comptes pour l'affichage.

Listing 5.2 – Provider de Comptes

```
class AccountProvider extends ChangeNotifier {
  final ApiService _apiService;
  final StorageService _storageService;

  List<BankAccount> _accounts = [];
  bool _isLoading = false;
  String? _error;

  AccountProvider({
    required ApiService apiService,
    required StorageService storageService,
  }) : _apiService = apiService,
       _storageService = storageService;

  List<BankAccount> get accounts => _accounts;
  bool get isLoading => _isLoading;
  String? get error => _error;

  Future<void> loadAccounts() async {
    _isLoading = true;
    _error = null;
    notifyListeners();

    try {
      final endpoint = await _isAdmin()
          ? '/api/accounts'
          : '/api/accounts/me';
      final response = await _apiService.get(endpoint);

      if (response.isSuccess) {
        final data = jsonDecode(response.data);
        final List<dynamic> accountsJson = data is List
            ? data
            : data['content'] ?? [];
```

```
        _accounts = accountsJson
            .map((json) => BankAccount.fromJson(json))
            .toList();
      } else {
        _error = 'Erreur lors du chargement des comptes';
      }
    } catch (e) {
      _error = 'Erreur: ${e.toString()}';
    } finally {
      _isLoading = false;
      notifyListeners();
    }
  }

  Future<bool> _isAdmin() async {
    final userInfo = await _storageService.getUserInfo();
    return userInfo != null &&
           (userInfo['roles']?.contains('ADMIN') == true ||
            userInfo['isAdmin'] == true);
  }
}
```

Ce provider adapte automatiquement l'endpoint API selon le rôle de l'utilisateur (admin ou client), garantissant que chaque utilisateur voit uniquement les données auxquelles il a accès.

5.2 Pattern Observer et Réactivité

Le pattern Provider utilise le pattern Observer pour notifier automatiquement tous les widgets qui écoutent les changements d'état. Lorsqu'un provider appelle `notifyListeners()`, tous les widgets qui utilisent `Consumer` ou `Provider.of` sont automatiquement reconstruits avec les nouvelles données.

Cette approche garantit que l'interface utilisateur reste toujours synchronisée avec l'état de l'application, sans nécessiter de gestion manuelle des mises à jour.

Écrans et Navigation

6.1 Système de Navigation

L'application utilise le système de navigation de Flutter avec des routes nommées pour gérer la navigation entre les écrans. Le système de routage est configuré dans `app_router.dart` et définit toutes les routes disponibles dans l'application.

6.1.1 Configuration des Routes

Les routes sont définies de manière centralisée pour faciliter la maintenance et éviter les erreurs de typage :

Listing 6.1 – Configuration des Routes

```
class AppRouter {
  static const String splash = '/';
  static const String login = '/login';
  static const String dashboard = '/dashboard';
  static const String adminDashboard = '/admin/dashboard';
  static const String accounts = '/accounts';
  static const String accountDetails = '/accounts/:id';
  static const String transactions = '/transactions';
  static const String customers = '/customers';
  static const String products = '/products';
  static const String leads = '/leads';
  static const String chat = '/chat';
  static const String profile = '/profile';
  static const String settings = '/settings';

  static Route<dynamic> generateRoute(RouteSettings settings) {
    switch (settings.name) {
      case splash:
        return MaterialPageRoute(builder: (_) => SplashScreen());
      case login:
        return MaterialPageRoute(builder: (_) => LoginScreen());
      case dashboard:
        return MaterialPageRoute(builder: (_) => DashboardScreen())
          ;
```

```
      case adminDashboard:
        return MaterialPageRoute(builder: (_) =>
           AdminDashboardScreen());
      // ... autres routes
      default:
        return MaterialPageRoute(
          builder: (_) => Scaffold(
            body: Center(child: Text('Route non trouv e')),
          ),
        );
    }
  }
}
```

Cette configuration centralisée facilite la maintenance et permet de gérer facilement les paramètres de route et les arguments passés entre les écrans.

6.2 Écrans Principaux

6.2.1 Écran de Connexion

L'écran de connexion (`LoginScreen`) est le point d'entrée de l'application pour les utilisateurs non authentifiés. Il offre une interface moderne et sécurisée pour la saisie des identifiants.

L'écran comprend un formulaire avec validation en temps réel, des messages d'erreur clairs, et un indicateur de chargement pendant l'authentification. Il gère également la redirection automatique vers le dashboard approprié après une connexion réussie (dashboard admin pour les administrateurs, dashboard client pour les clients).

L'interface utilise Material Design 3 avec des champs de texte modernes, des boutons avec des animations de feedback, et une gestion élégante des erreurs avec des messages contextuels.

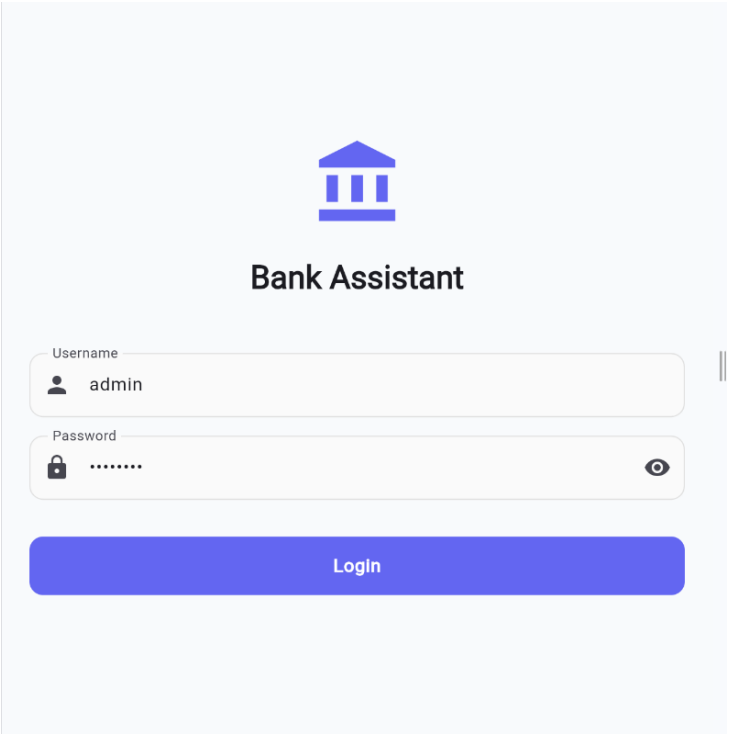


FIGURE 6.1 – Écran de connexion - Interface moderne avec validation en temps réel

6.2.2 Dashboard Client

Le dashboard client (`CustomerDashboardScreen`) offre une vue personnalisée pour les clients du système bancaire. Il affiche leurs comptes, leurs transactions récentes, et leurs produits financiers.

screens/customer_dashboard.png

FIGURE 6.2 – Dashboard client - Vue personnalisée pour les clients

6.2.3 Dashboard Administrateur

Le dashboard administrateur (`AdminDashboardScreen`) offre une vue d'ensemble complète du système bancaire. Il affiche des statistiques en temps réel, des graphiques de performance, et des indicateurs clés de performance (KPI).

Le dashboard est organisé en plusieurs sections :

— **Vue d'ensemble** : Statistiques globales (nombre de clients, comptes, transactions, revenus)

— **Actions rapides** : Boutons d'accès rapide aux fonctionnalités principales

— **Graphiques** : Visualisations des tendances et des performances

— **Notifications** : Alertes et notifications importantes

L'interface est conçue pour être informative sans être surchargée, avec une hiérarchie visuelle claire et des couleurs qui guident l'attention de l'utilisateur vers les informations les plus importantes.

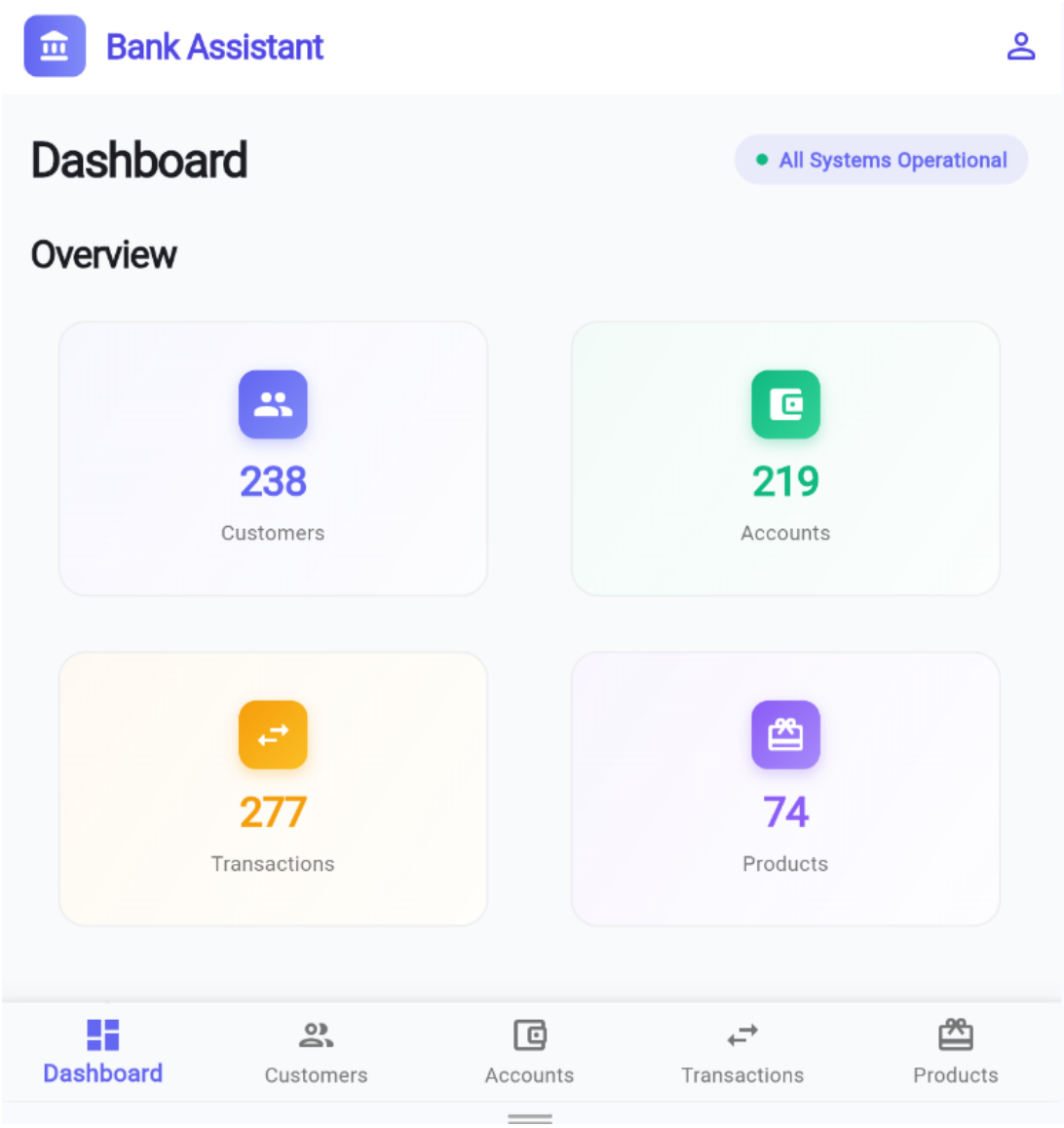


FIGURE 6.3 – Dashboard administrateur - Vue d'ensemble complète du système bancaire

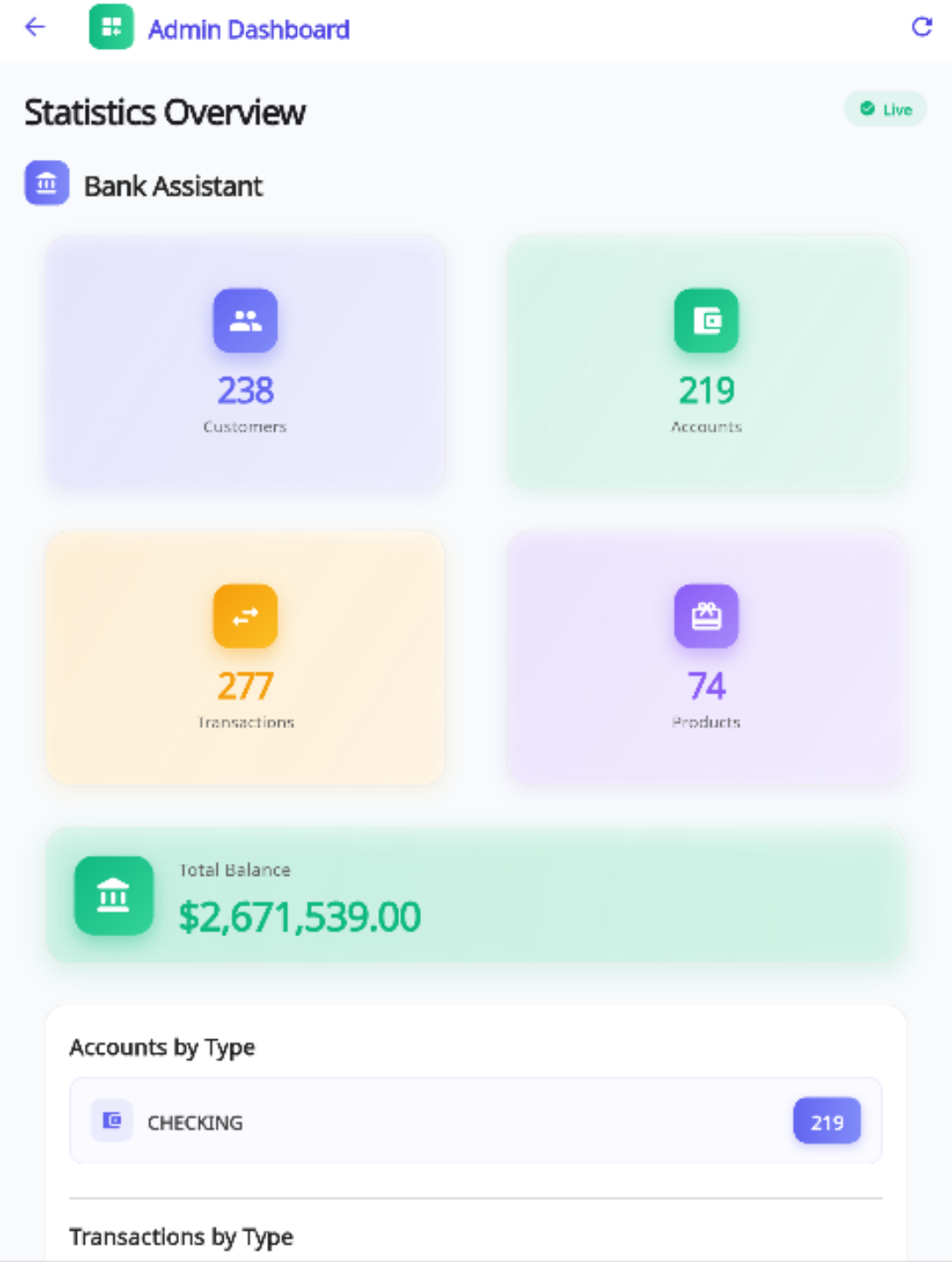


FIGURE 6.4 – Dashboard administrateur - Section statistiques et indicateurs de performance

6.2.4 Écran de Gestion des Comptes

L'écran de gestion des comptes (`AccountsScreen`) affiche la liste de tous les comptes bancaires. Pour les administrateurs, il affiche tous les comptes du système avec des options de filtrage et de recherche. Pour les clients, il affiche uniquement leurs propres comptes.

L'écran comprend :

— Une barre de recherche pour filtrer les comptes
— Des filtres par type de compte, statut, et devise
— Une liste scrollable avec pagination automatique
— Des cartes de compte avec les informations essentielles
— Un bouton flottant pour créer un nouveau compte (admin uniquement)

Chaque carte de compte affiche le numéro de compte, le type, le solde formaté avec la devise, le nom du client (pour les admins), et le statut du compte. Un tap sur une carte ouvre l'écran de détails du compte.

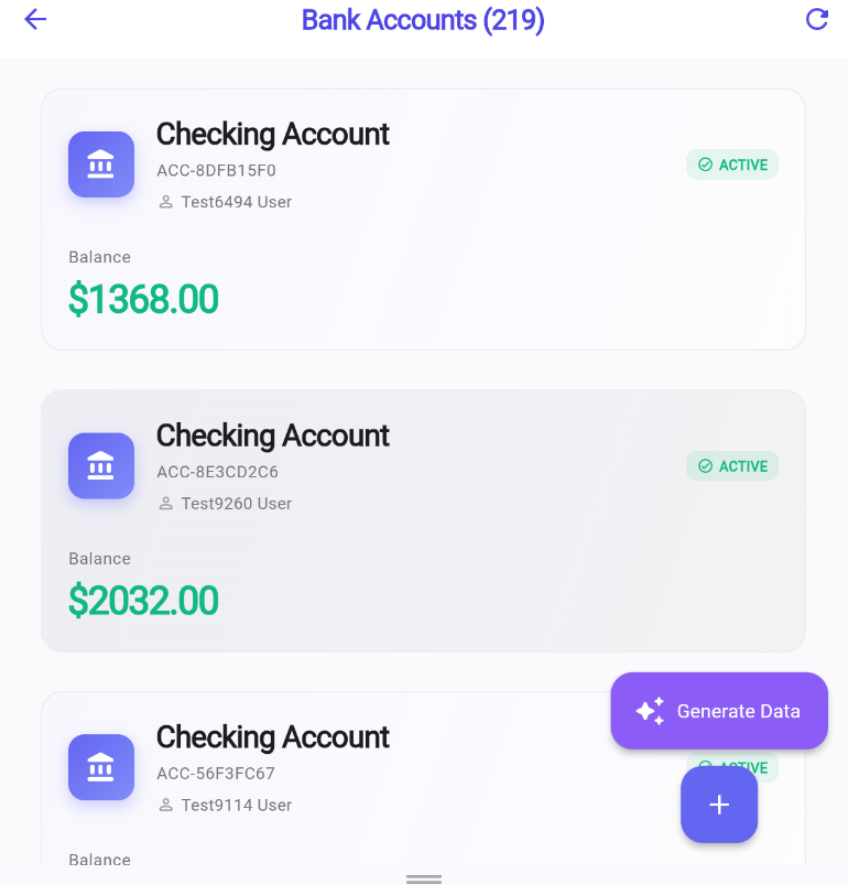


FIGURE 6.5 – Écran de gestion des comptes - Liste des comptes avec filtres et recherche

6.2.5 Écran de Détails de Compte

L'écran de détails de compte (`AccountDetailsScreen`) affiche toutes les informations détaillées d'un compte spécifique. Il comprend :

— **Informations générales** : Numéro de compte, type, solde, devise, statut
— **Informations client** : Nom du client, email, numéro de client
— **Historique des transactions** : Liste des transactions récentes avec pagination
— **Actions** : Boutons pour effectuer des opérations (dépôt, retrait, transfert)

L'écran utilise des widgets réutilisables (`DetailCard`) pour afficher les informations de manière cohérente et lisible. Les transactions sont affichées dans une liste avec des icônes colorées selon le type de transaction (dépôt en vert, retrait en rouge, transfert en bleu).

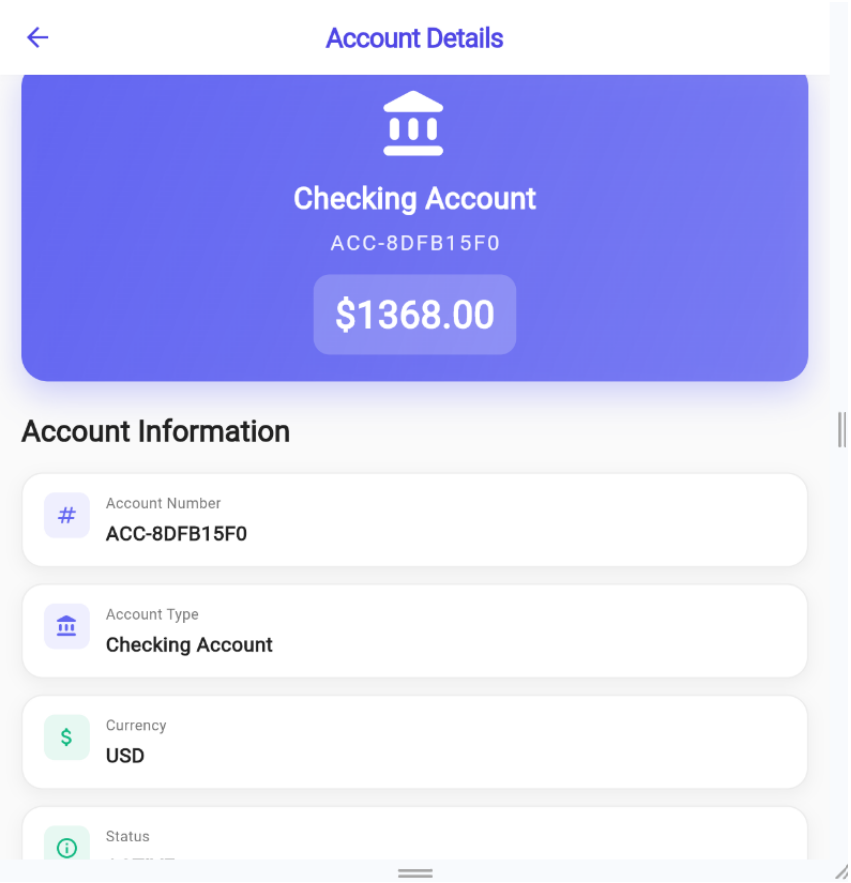


FIGURE 6.6 – Écran de détails de compte - Informations complètes et historique des transactions

6.2.6 Écran de Gestion des Clients

L'écran de gestion des clients (`CustomersScreen`) permet aux administrateurs de consulter et gérer tous les clients du système. Il affiche la liste des clients avec leurs informations principales et permet la recherche et le filtrage.

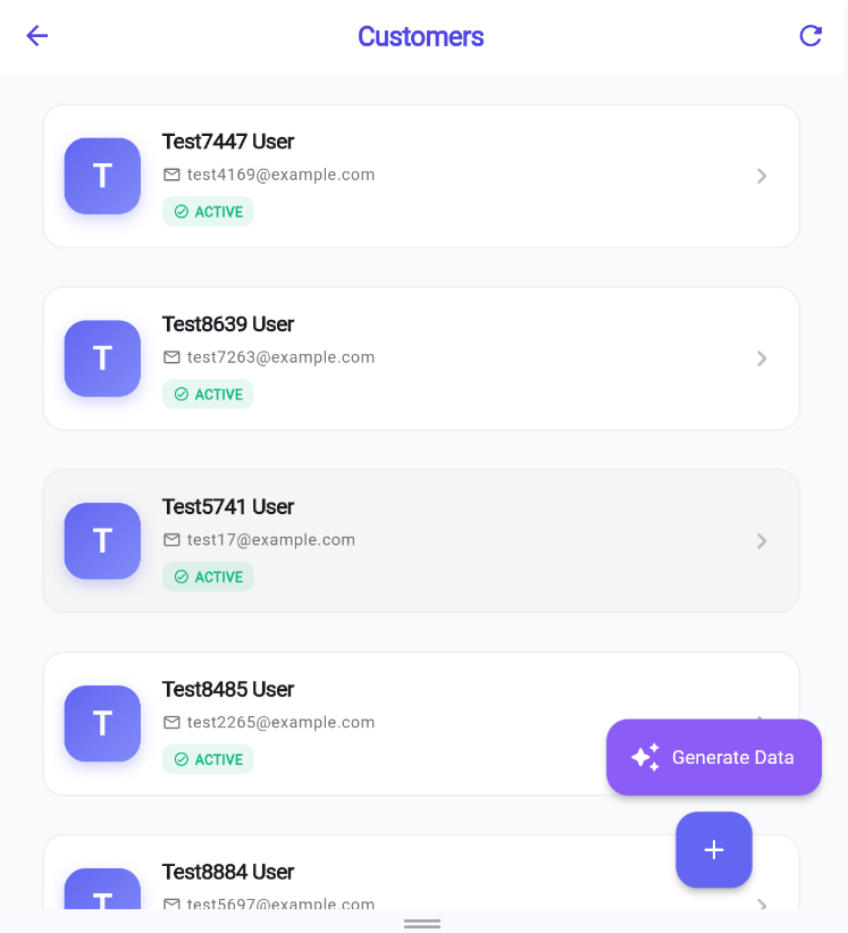


FIGURE 6.7 – Écran de gestion des clients - Liste complète avec recherche et filtres

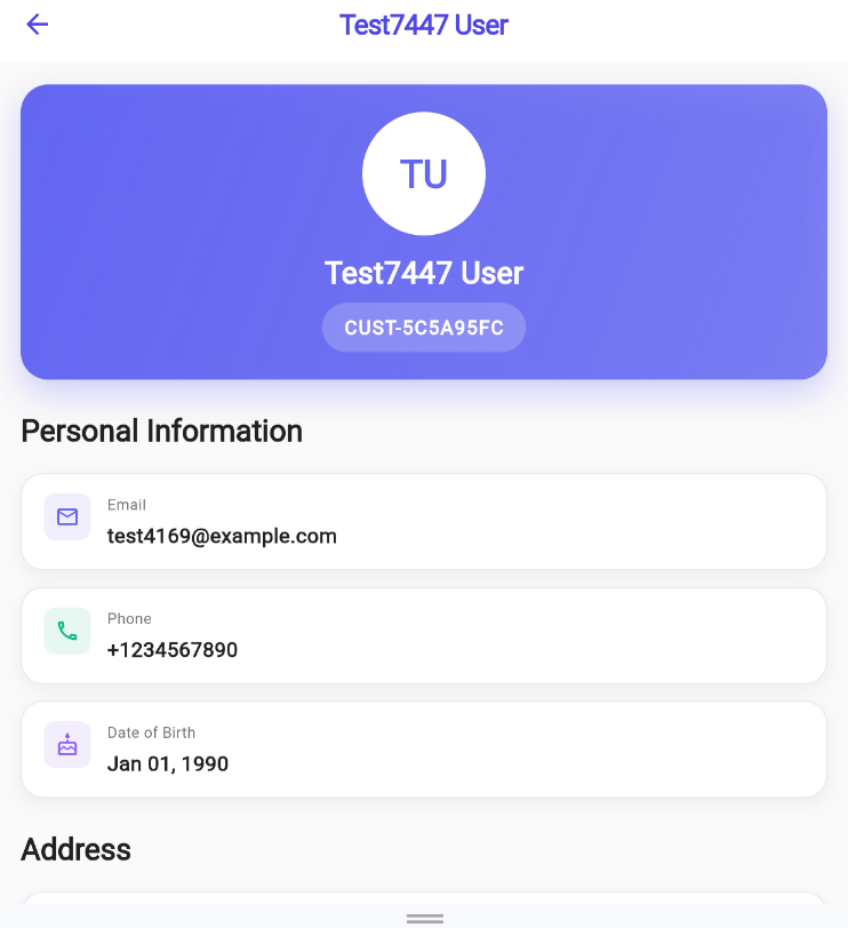


FIGURE 6.8 – Écran de détails de client - Informations complètes du client

6.2.7 Écran de Gestion des Transactions

L'écran de gestion des transactions (`TransactionsScreen`) affiche toutes les transactions du système (pour les admins) ou les transactions de l'utilisateur connecté (pour les clients).

L'écran comprend :

- Des filtres par type de transaction, date, et montant
- Une recherche par numéro de transaction ou description
- Une liste de transactions avec pagination
- Des indicateurs visuels pour les différents types de transactions
- Un bouton pour créer une nouvelle transaction (admin uniquement)

Chaque transaction est affichée dans une carte avec le type, le montant formaté, la date, le compte associé, et le nom du client (pour les admins). Les montants sont colorés selon le type : vert pour les dépôts, rouge pour les retraits.

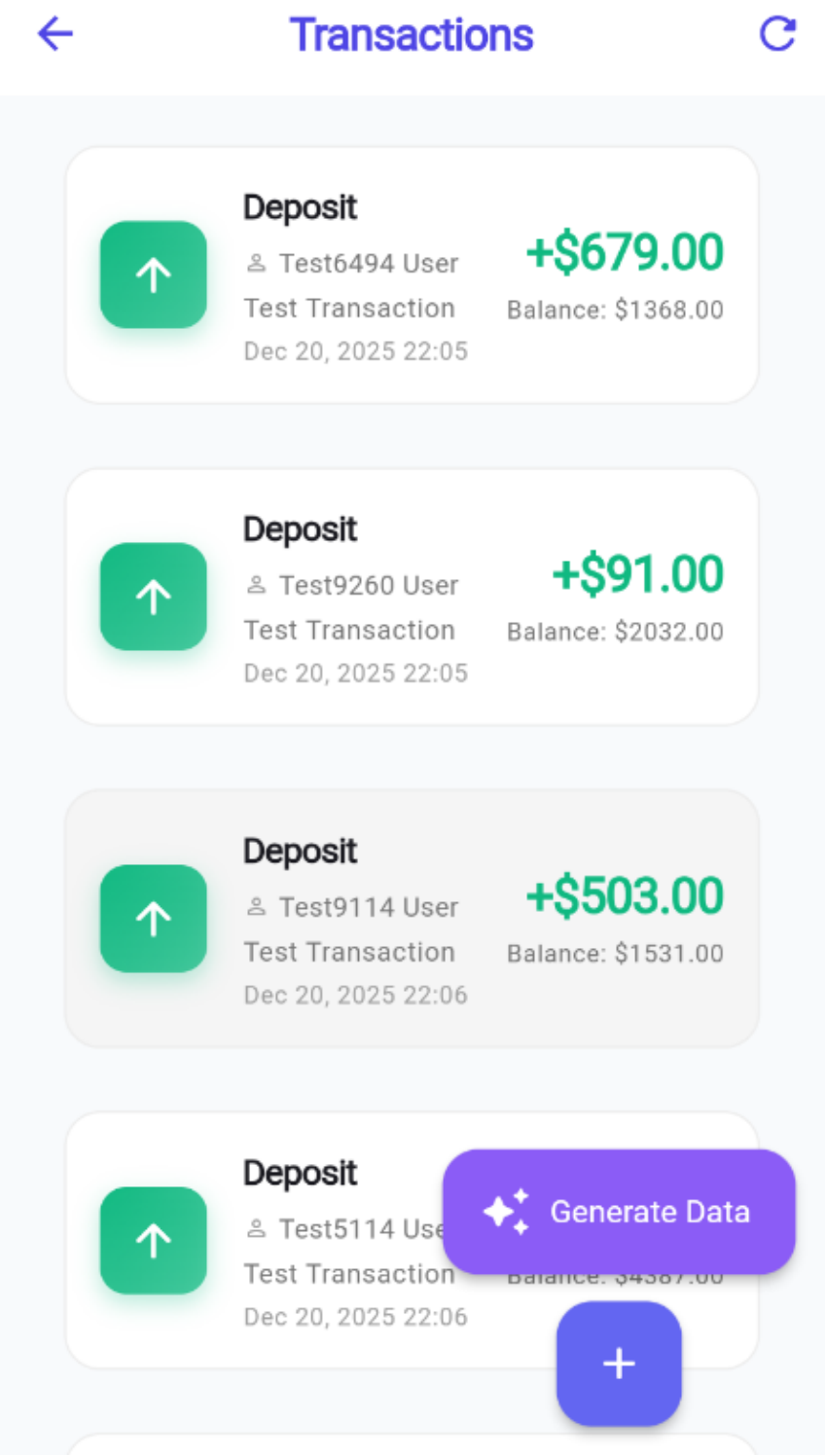


FIGURE 6.9 – Écran de gestion des transactions - Liste avec filtres et indicateurs visuels

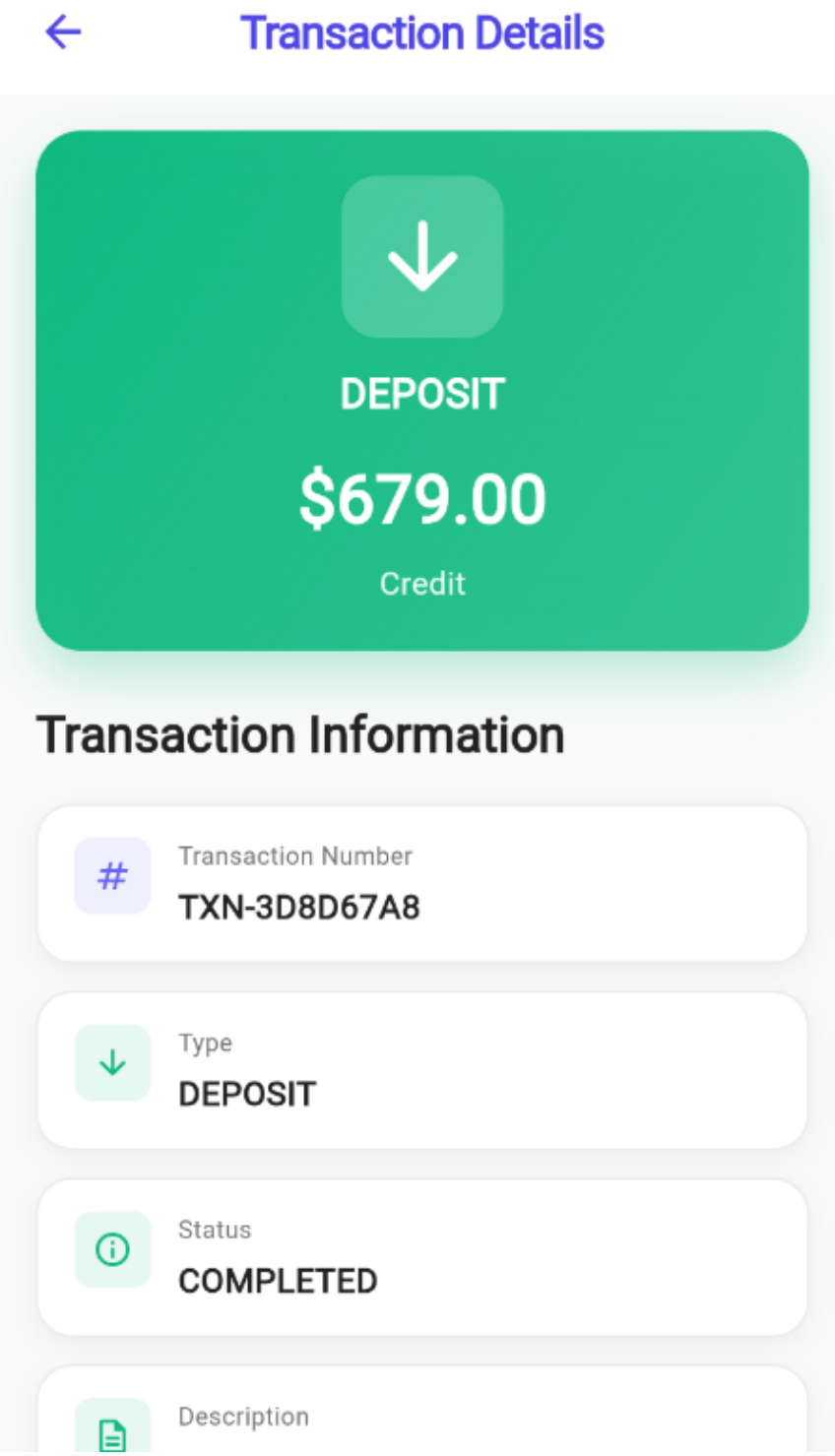


FIGURE 6.10 – Écran de détails de transaction - Informations complètes de la transaction

6.2.8 Écran de Chat

L'écran de chat (`ChatScreen`) permet aux utilisateurs d'interagir avec l'assistant virtuel intelligent. Il offre une interface de chat moderne similaire aux applications de mes-

sagerie populaires.

L'écran comprend :

— Une zone de messages scrollable affichant l'historique de la conversation
— Un champ de saisie avec un bouton d'envoi
— Des indicateurs de frappe en cours
— Des animations fluides pour l'apparition des nouveaux messages
— Une gestion de l'état de chargement pendant l'envoi et la réception des messages

Les messages de l'utilisateur sont alignés à droite avec un style distinct, tandis que les messages de l'assistant sont alignés à gauche. L'interface gère automatiquement le scroll vers le bas lors de l'arrivée de nouveaux messages.

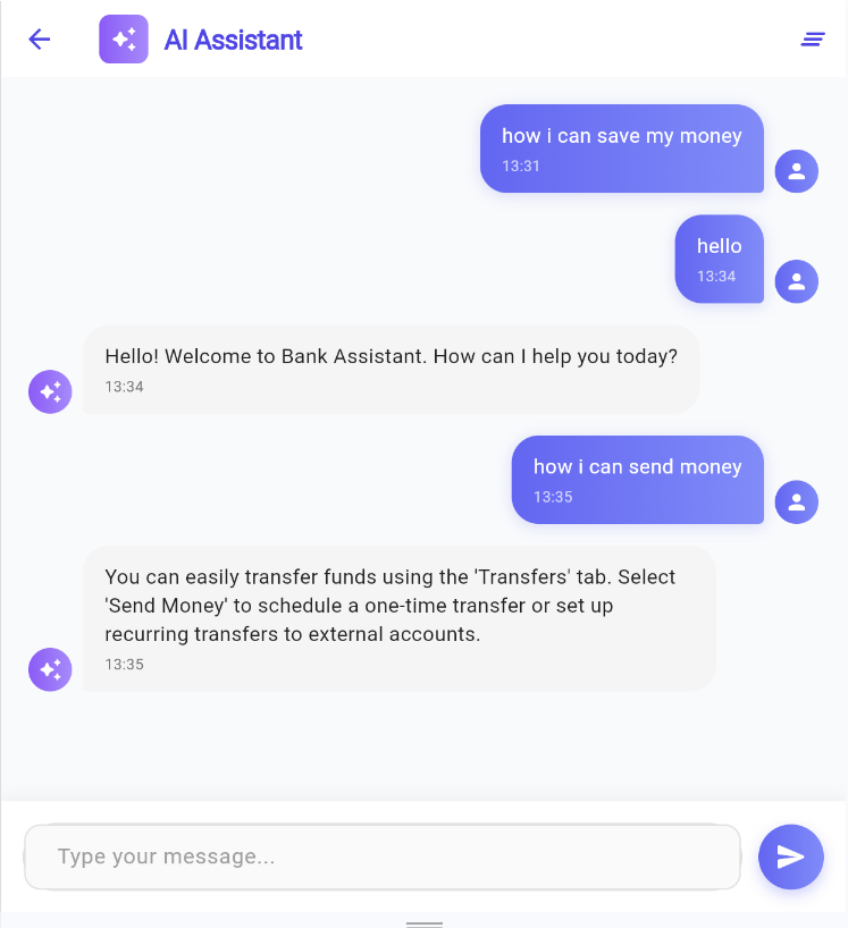


FIGURE 6.11 – Écran de chat - Interface de messagerie avec l'assistant virtuel intelligent

Widgets Réutilisables et Composants UI

7.1 Design System

L'application utilise un design system cohérent basé sur Material Design 3 pour garantir une expérience utilisateur uniforme dans toute l'application. Le design system définit les couleurs, les typographies, les espacements, et les composants réutilisables.

7.1.1 Thème de l'Application

Le thème de l'application est défini dans `app_theme.dart` et utilise Material Design 3 avec une palette de couleurs adaptée au secteur bancaire :

Listing 7.1 – Configuration du Thème

```
class AppTheme {
  static ThemeData get lightTheme {
    return ThemeData(
      useMaterial3: true,
      colorScheme: ColorScheme.fromSeed(
        seedColor: Colors.blue,
        brightness: Brightness.light,
      ),
      cardTheme: CardThemeData(
        elevation: 2,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(12),
        ),
      ),
      inputDecorationTheme: InputDecorationTheme(
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(8),
        ),
        filled: true,
      ),
    );
  }
}
```

Le thème définit les styles de base pour les cartes, les champs de formulaire, les boutons, et autres composants, garantissant une apparence cohérente dans toute l'application.

7.1.2 Widget ModernCard

Le widget `ModernCard` est un composant réutilisable qui crée des cartes modernes avec des ombres douces, des bordures arrondies, et des effets visuels élégants. La figure 7.1 illustre l'utilisation de ce widget dans l'interface.

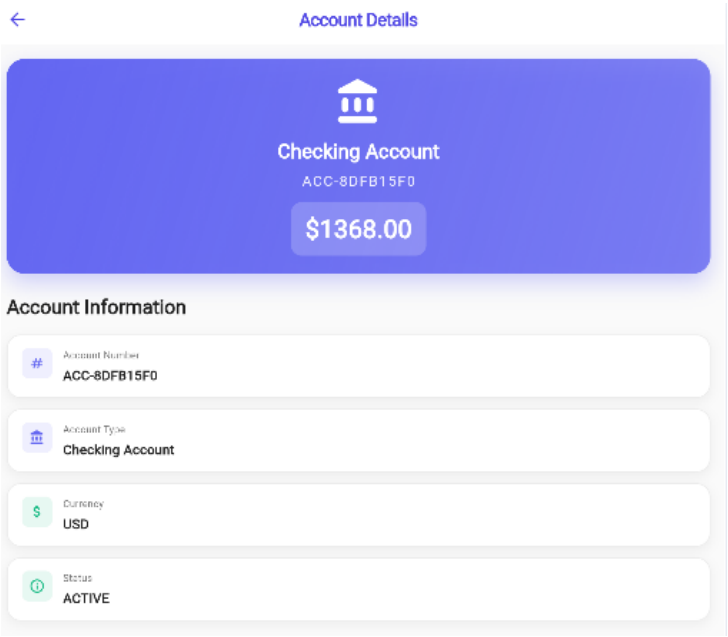


FIGURE 7.1 – Exemple d'utilisation du widget ModernCard - Cartes avec design moderne

Listing 7.2 – Widget ModernCard

```
class ModernCard extends StatelessWidget {
  final Widget child;
  final EdgeInsets? padding;
  final Color? backgroundColor;
  final VoidCallback? onTap;

  const ModernCard({
    Key? key,
    required this.child,
    this.padding,
    this.backgroundColor,
    this.onTap,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Card(
      elevation: 2,
```

```
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
      ),
      child: InkWell(
        onTap: onTap,
        borderRadius: BorderRadius.circular(12),
        child: Container(
          padding: padding ?? EdgeInsets.all(16),
          decoration: BoxDecoration(
            color: backgroundColor ?? Colors.white,
            borderRadius: BorderRadius.circular(12),
          ),
          child: child,
        ),
      ),
    );
  }
}
```

Ce widget est utilisé dans toute l'application pour créer des cartes cohérentes et modernes, améliorant ainsi l'esthétique globale de l'interface.

7.1.3 Widget DetailCard

Le widget `DetailCard` est spécialement conçu pour afficher des paires clé-valeur dans les écrans de détails :

Listing 7.3 – Widget DetailCard

```
class DetailCard extends StatelessWidget {
  final String label;
  final String value;
  final IconData? icon;

  const DetailCard({
    Key? key,
    required this.label,
    required this.value,
    this.icon,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return ModernCard(
      padding: EdgeInsets.all(16),
      child: Row(
        children: [
          if (icon != null) ...[
            Icon(icon, color: Colors.blue),
            SizedBox(width: 12),
```

```
          ],
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  label,
                  style: TextStyle(
                    fontSize: 12,
                    color: Colors.grey[600],
                  ),
                ),
                SizedBox(height: 4),
                Text(
                  value,
                  style: TextStyle(
                    fontSize: 16,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

Ce widget garantit une présentation cohérente des informations dans tous les écrans de détails, avec des icônes optionnelles pour améliorer la lisibilité.

7.1.4 Widget ModernModal

Le widget `ModernModal` crée des modales modernes et élégantes pour les formulaires et les dialogues. La figure 7.2 montre un exemple de modal utilisé pour la création de données.

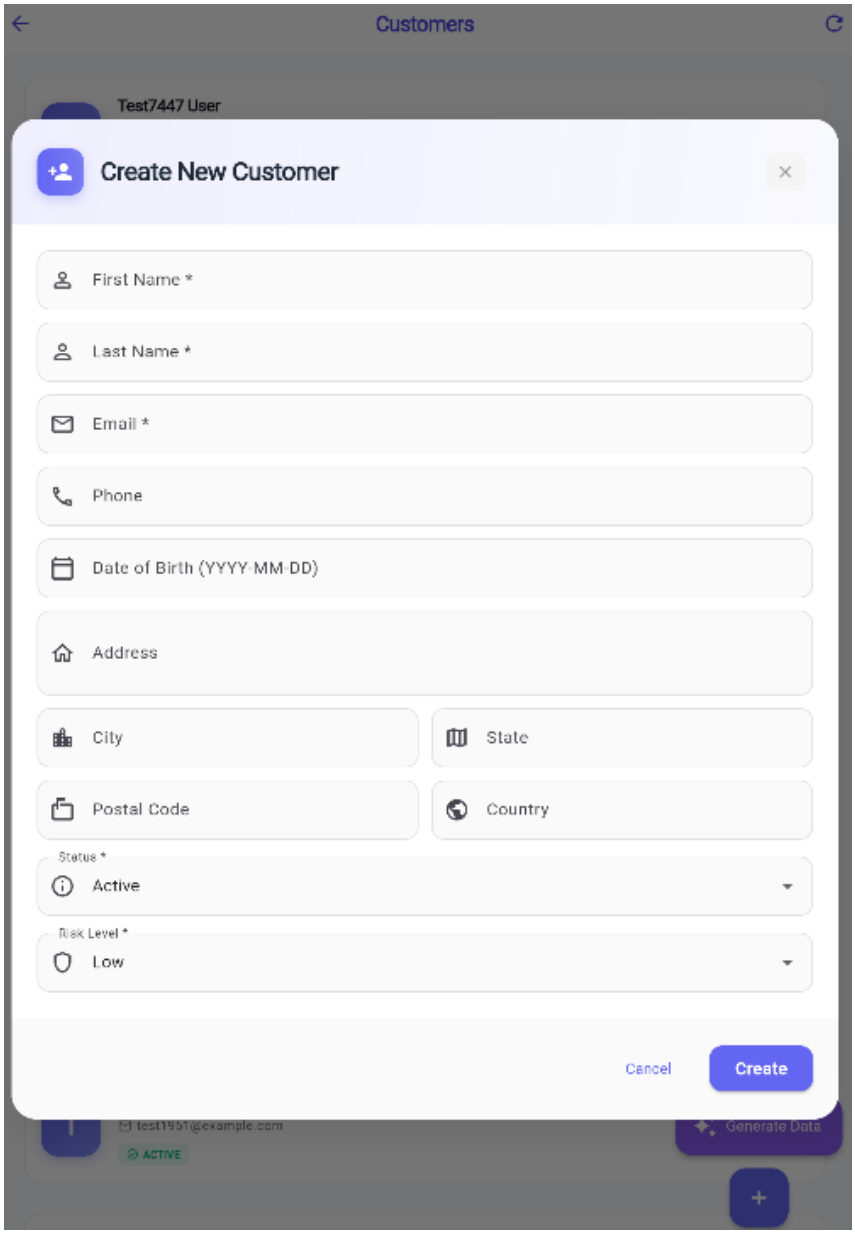


FIGURE 7.2 – Exemple d'utilisation du widget ModernModal - Modal moderne pour formulaires

Listing 7.4 – Widget ModernModal

```
class ModernModal extends StatelessWidget {
  final String title;
  final Widget content;
  final List<Widget> actions;
  final bool isDismissible;

  const ModernModal({
    Key? key,
    required this.title,
    required this.content,
```

```
    required this.actions,
    this.isDismissible = true,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Dialog(
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(16),
      ),
      child: Container(
        padding: EdgeInsets.all(24),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              title,
              style: TextStyle(
                fontSize: 20,
                fontWeight: FontWeight.bold,
              ),
            ),
            SizedBox(height: 16),
            content,
            SizedBox(height: 24),
            Row(
              mainAxisAlignment: MainAxisAlignment.end,
              children: actions,
            ),
          ],
        ),
      ),
    );
  }
}
```

Ce widget est utilisé pour tous les formulaires de création et de modification, garantissant une expérience utilisateur cohérente et moderne.

Expérience Utilisateur et Interface

8.1 Design Moderne et Professionnel

L'interface utilisateur de l'application a été conçue pour être moderne, douce et professionnelle, adaptée au secteur bancaire. Le design utilise des gradients subtils, des ombres douces, des bordures arrondies, et une palette de couleurs professionnelle.

Les couleurs principales sont basées sur le bleu, couleur traditionnellement associée à la confiance et à la stabilité dans le secteur bancaire. Les accents de couleur sont utilisés avec parcimonie pour guider l'attention de l'utilisateur vers les actions importantes.

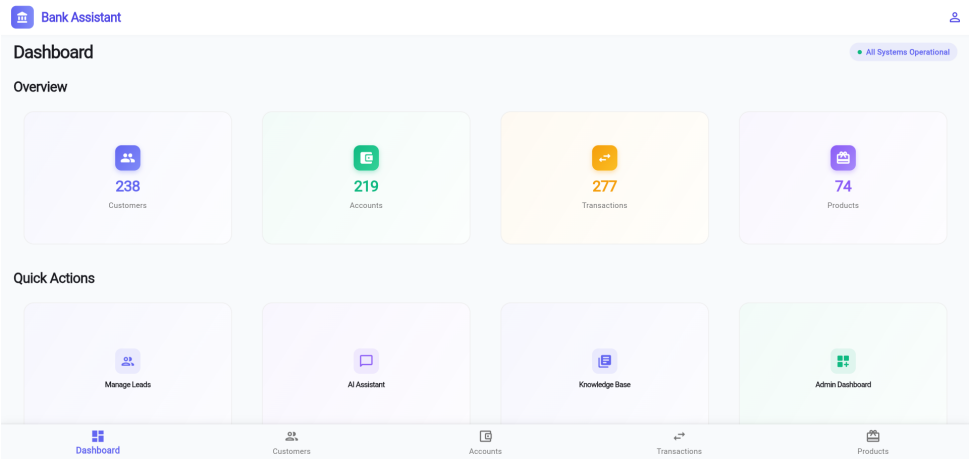


FIGURE 8.1 – Vue d'ensemble du design - Interface moderne et professionnelle

8.1.1 Hierarchie Visuelle

La hiérarchie visuelle est soigneusement conçue pour guider l'utilisateur à travers l'interface. Les éléments les plus importants sont mis en évidence par leur taille, leur couleur, ou leur position. Les titres utilisent des tailles de police différentes pour créer une hiérarchie claire, et les espacements sont cohérents pour créer un rythme visuel agréable.

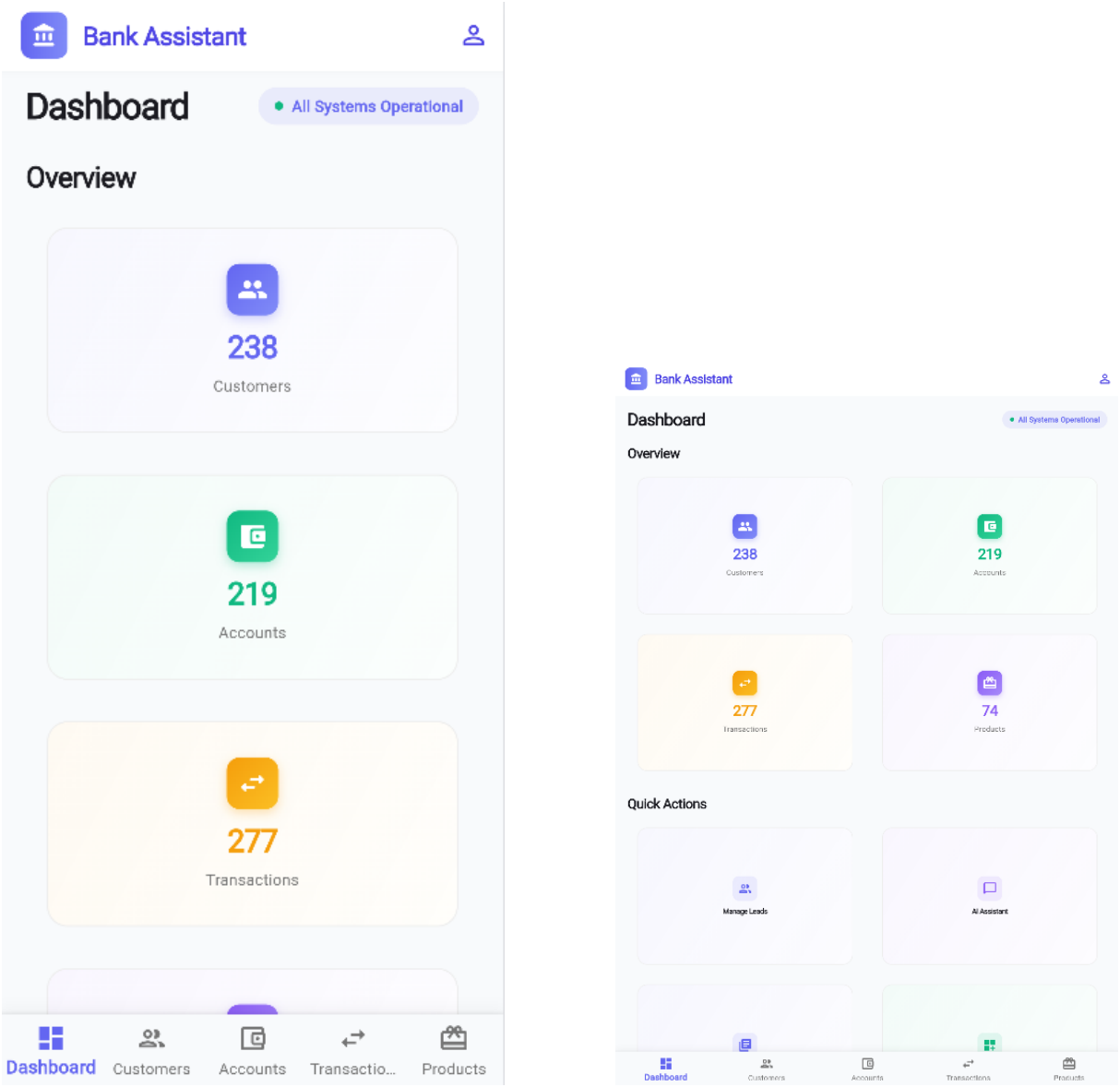
8.1.2 Feedback Utilisateur

L'application fournit un feedback immédiat pour toutes les actions utilisateur :

— **Animations** : Transitions fluides entre les écrans et animations de feedback pour les interactions
— **Indicateurs de chargement** : Spinners et skeletons pendant le chargement des données
— **Messages de succès/erreur** : SnackBars et dialogs pour informer l'utilisateur du résultat des actions
— **États de validation** : Feedback visuel en temps réel dans les formulaires

8.2 Responsivité et Adaptabilité

L'application est conçue pour s'adapter à différentes tailles d'écran, des petits smartphones aux grandes tablettes. La responsivité est gérée à plusieurs niveaux :



(a) Vue mobile

(b) Vue tablette

FIGURE 8.2 – Responsivité de l'application - Adaptation aux différentes tailles d'écran

Layouts Adaptatifs : Les layouts utilisent `LayoutBuilder` et `MediaQuery` pour s'adapter à la taille de l'écran. Sur les petits écrans, les éléments sont empilés verticalement, tandis que sur les grands écrans, ils peuvent être disposés horizontalement.

Grilles Adaptatives : Les grilles utilisent `GridView.count` avec un nombre de colonnes adaptatif selon la largeur de l'écran. Sur les petits écrans, une seule colonne est affichée, tandis que sur les grands écrans, plusieurs colonnes peuvent être affichées.

Widgets Responsifs : Les widgets réutilisables comme `DetailCard` et `ModernCard` s'adaptent automatiquement à la taille disponible, garantissant une présentation optimale sur tous les appareils.

8.3 Gestion des États de Chargement

L'application gère élégamment les différents états de chargement pour offrir une expérience utilisateur fluide :

État Initial : Lors du premier chargement, un indicateur de chargement est affiché avec un message informatif.

État de Chargement : Pendant le chargement des données, des skeletons ou des spinners sont affichés pour indiquer que l'application travaille.

État de Succès : Une fois les données chargées, elles sont affichées avec une animation d'apparition fluide.

État d'Erreur : En cas d'erreur, un message d'erreur clair est affiché avec un bouton pour réessayer.

État Vide : Lorsqu'aucune donnée n'est disponible, un message informatif est affiché avec des suggestions d'actions.

Sécurité et Authentification

9.1 Gestion de l'Authentification

L'authentification dans l'application est gérée de manière sécurisée avec plusieurs couches de protection :

Stockage Sécurisé des Tokens : Les tokens JWT sont stockés de manière sécurisée dans `shared_preferences` avec des clés cryptées. Les tokens ne sont jamais stockés en clair et sont automatiquement supprimés lors de la déconnexion.

Validation des Tokens : Avant chaque requête API, le token est validé et rafraîchi si nécessaire. Si le token est expiré, l'utilisateur est automatiquement déconnecté et redirigé vers l'écran de connexion.

Gestion des Rôles : L'application gère les différents rôles (ADMIN, CUSTOMER) et adapte l'interface et les fonctionnalités disponibles selon le rôle de l'utilisateur connecté.

9.2 Contrôle d'Accès

L'application implémente un contrôle d'accès basé sur les rôles (RBAC) pour garantir que chaque utilisateur ne peut accéder qu'aux fonctionnalités autorisées :

Protection des Routes : Les routes sensibles sont protégées et vérifient l'authentification et les autorisations avant d'autoriser l'accès.

Adaptation de l'Interface : L'interface s'adapte automatiquement selon le rôle de l'utilisateur, masquant les fonctionnalités non autorisées.

Validation Côté Client : Bien que la validation principale soit effectuée côté serveur, l'application effectue également des validations côté client pour améliorer l'expérience utilisateur et réduire les appels API inutiles.

Performance et Optimisations

10.1 Optimisations de Performance

L'application a été optimisée pour offrir des performances optimales même sur des appareils moins puissants :

Lazy Loading : Les listes utilisent le lazy loading pour charger les données progressivement au fur et à mesure du scroll, réduisant ainsi la consommation mémoire et améliorant les temps de chargement initiaux.

Caching : Les données fréquemment accédées sont mises en cache localement pour réduire les appels API et améliorer la réactivité de l'application.

Optimisation des Images : Les images sont optimisées et chargées de manière asynchrone pour ne pas bloquer l'interface utilisateur.

Débouncing : Les recherches et les filtres utilisent le débouncing pour éviter les appels API excessifs pendant la saisie utilisateur.

10.2 Gestion de la Mémoire

La gestion de la mémoire est optimisée pour éviter les fuites mémoire et garantir des performances stables :

Disposition des Contrôleurs : Tous les contrôleurs (TextEditingController, AnimationController, etc.) sont correctement disposés lorsqu'ils ne sont plus nécessaires.

Éviter les Rebuilds Inutiles : Les widgets sont optimisés pour éviter les rebuilds inutiles en utilisant `const` constructors et `shouldRebuild` dans les providers.

Nettoyage des Listeners : Tous les listeners sont correctement supprimés pour éviter les fuites mémoire.

Gestion des Erreurs et Robustesse

11.1 Stratégie de Gestion des Erreurs

L'application implémente une stratégie complète de gestion des erreurs pour garantir une expérience utilisateur fluide même en cas de problèmes :

Erreurs Réseau : Les erreurs de connexion réseau sont détectées et affichées avec des messages clairs et des options pour réessayer.

Erreurs de Validation : Les erreurs de validation sont affichées directement dans les formulaires avec des messages spécifiques pour chaque champ.

Erreurs Serveur : Les erreurs serveur (500, 503, etc.) sont gérées avec des messages utilisateur compréhensibles et des options de récupération.

Erreurs d'Authentification : Les erreurs d'authentification déclenchent automatiquement une déconnexion et une redirection vers l'écran de connexion.

11.2 Logging et Debugging

L'application utilise un système de logging structuré pour faciliter le debugging et le monitoring :

Logs Structurés : Les logs sont structurés avec des niveaux appropriés (DEBUG, INFO, WARNING, ERROR) pour faciliter le filtrage et l'analyse.

Mode Debug : En mode debug, des logs détaillés sont affichés pour aider au développement, tandis qu'en mode production, seuls les logs essentiels sont conservés.

Reporting d'Erreurs : Les erreurs critiques peuvent être rapportées automatiquement pour permettre une correction rapide.

Conclusion

12.1 Résumé des Réalisations

Ce rapport a présenté une analyse complète de l'application frontend Flutter développée pour le système Bank Assistant. Les réalisations principales incluent :

- Une architecture modulaire et scalable organisée en couches claires
- Un système de gestion d'état robuste avec Provider
- Une interface utilisateur moderne et professionnelle adaptée au secteur bancaire
- Une gestion complète de l'authentification et de la sécurité
- Des widgets réutilisables pour garantir la cohérence de l'interface
- Une gestion élégante des erreurs et des états de chargement
- Des optimisations de performance pour une utilisation fluide
- Une responsivité complète pour différentes tailles d'écran

12.2 Points Forts de l'Application

Les points forts principaux de l'application frontend sont :

Expérience Utilisateur Exceptionnelle : L'interface est moderne, intuitive, et offre un feedback immédiat pour toutes les actions utilisateur. La navigation est fluide et les transitions sont élégantes.

Architecture Robuste : L'architecture modulaire facilite la maintenance, les tests, et les évolutions futures. La séparation des responsabilités garantit un code propre et maintenable.

Sécurité Renforcée : La gestion sécurisée de l'authentification et le contrôle d'accès basé sur les rôles garantissent que seuls les utilisateurs autorisés peuvent accéder aux fonctionnalités appropriées.

Performance Optimale : Les optimisations de performance garantissent une utilisation fluide même sur des appareils moins puissants, avec des temps de chargement rapides et une consommation mémoire optimisée.

12.3 Perspectives d'Amélioration

Pour continuer à améliorer l'application, plusieurs axes d'amélioration sont identifiés :

1. **Internationalisation** : Ajouter le support de plusieurs langues pour rendre l'application accessible à un public international
2. **Mode Hors Ligne** : Implémenter un mode hors ligne avec synchronisation automatique lors de la reconnexion
3. **Notifications Push** : Ajouter les notifications push pour informer les utilisateurs des événements importants
4. **Biométrie** : Implémenter l'authentification biométrique pour améliorer la sécurité et la commodité
5. **Thème Sombre** : Ajouter un thème sombre pour améliorer le confort visuel et économiser la batterie
6. **Accessibilité** : Améliorer l'accessibilité pour les utilisateurs avec des besoins spéciaux
7. **Tests Automatisés** : Ajouter des tests unitaires et d'intégration pour garantir la qualité du code

12.4 Engagement de l'Équipe

L'équipe s'engage à maintenir et améliorer continuellement l'application frontend en :

— Respectant les meilleures pratiques de développement Flutter
— Maintenant une architecture propre et modulaire
— Améliorant constamment l'expérience utilisateur
— Optimisant les performances et la consommation de ressources
— Garantissant la sécurité et la protection des données utilisateur
— Documentant le code et les décisions architecturales
— Partageant les connaissances et les bonnes pratiques

Document approuvé par l'équipe de développement

Date d'approbation : 22 décembre 2025

Osama Mansouri	22 décembre 2025
Sohaib Laarichi	22 décembre 2025
Salah Eddine Aboulkameh	22 décembre 2025
Yassir Rabai	22 décembre 2025
Hanae Sbai (Encadrant)	22 décembre 2025

Annexe A : Liste des Captures d'Écran

Cette annexe liste toutes les captures d'écran nécessaires pour illustrer le rapport. Toutes les images doivent être placées dans le répertoire `doc/screens/` avec les noms suivants :

Nom du Fichier	Description	Section
`login_screen.png`	Écran de connexion avec formulaire	Chapitre 6 - Écran de Connexion
`admin_dashboard.png`	Dashboard administrateur - vue principale	Chapitre 6 - Dashboard Admin
`admin_dashboard_stats.png`	Dashboard administrateur - section statistiques	Chapitre 6 - Dashboard Admin
`customer_dashboard.png`	Dashboard client - vue personnalisée	Chapitre 6 - Dashboard Client
`accounts_list.png`	Liste des comptes avec filtres	Chapitre 6 - Gestion Comptes
`account_details.png`	Détails d'un compte avec transactions	Chapitre 6 - Détails Compte
`transactions_list.png`	Liste des transactions	Chapitre 6 - Gestion Transactions
`transaction_details.png`	Détails d'une transaction	Chapitre 6 - Détails Transaction
`customers_list.png`	Liste des clients	Chapitre 6 - Gestion Clients
`customer_details.png`	Détails d'un client	Chapitre 6 - Détails Client
`chat_screen.png`	Interface de chat avec assistant	Chapitre 6 - Écran de Chat
`modern_card_example.png`	Exemple d'utilisation ModernCard	Chapitre 7 - Widget ModernCard
`modern_modal_example.png`	Exemple de modal moderne	Chapitre 7 - Widget ModernModal
`design_overview.png`	Vue d'ensemble du design	Chapitre 8 - Design Moderne
`responsive_mobile.png`	Vue sur mobile	Chapitre 8 - Responsivité
`responsive_tablet.png`	Vue sur tablette	Chapitre 8 - Responsivité

Table A.1 – Liste des Captures d'Écran Requises

Instructions pour les captures d'écran :

- Format : PNG avec résolution minimale de 1080x1920 pixels (portrait) ou 1920x1080 pixels (paysage)
- Qualité : Haute résolution pour une impression claire
- Contenu : Capturer l'écran complet avec les données réelles de l'application
- Nommage : Respecter exactement les noms de fichiers indiqués dans le tableau
- Emplacement : Toutes les images doivent être dans `doc/screens/`

Annexe B : Structure Complète du Projet

```
frontend/
 lib/
    core/
       config/
          app_config.dart
       models/
          bank_account.dart
          bank_account.g.dart
          customer.dart
          customer.g.dart
          transaction.dart
          transaction.g.dart
          financial_product.dart
          financial_product.g.dart
          lead.dart
          lead.g.dart
          knowledge_base.dart
          knowledge_base.g.dart
       providers/
          auth_provider.dart
          account_provider.dart
          transaction_provider.dart
          customer_provider.dart
          product_provider.dart
          lead_provider.dart
          chat_provider.dart
          admin_provider.dart
          knowledge_base_provider.dart
       services/
          api_service.dart
          auth_service.dart
          storage_service.dart
```

```
        fake_data_service.dart
    routes/
        app_router.dart
    theme/
        app_theme.dart
    utils/
         responsive.dart
screens/
    splash_screen.dart
    auth/
        login_screen.dart
        profile_screen.dart
    admin/
        admin_dashboard_screen.dart
        knowledge_base_screen.dart
    banking/
        dashboard_screen.dart
        accounts_screen.dart
        transactions_screen.dart
        customers_screen.dart
        products_screen.dart
        details/
            account_details_screen.dart
            transaction_details_screen.dart
            customer_details_screen.dart
            product_details_screen.dart
    customer/
        customer_dashboard_screen.dart
        customer_profile_screen.dart
        my_accounts_screen.dart
        my_transactions_screen.dart
        details/
            my_account_details_screen.dart
            my_transaction_details_screen.dart
    leads/
        leads_screen.dart
        details/
            lead_details_screen.dart
    chat/
        chat_screen.dart
    settings/
         settings_screen.dart
widgets/
    modern_card.dart
    detail_card.dart
    modern_modal.dart
```

```
        fab_menu.dart
    main.dart
pubspec.yaml
README.md
```

Annexe C : Liste des Packages et Versions

Package	**Version**
flutter	SDK
provider	^{6}.1.1
http	^{1}.2.0
json_annotation	^{4}.8.1
json_serializable	^{6}.7.1
build_runner	^{2}.4.7
shared_preferences	^{2}.2.2
intl	^{0}.19.0
flutter_dotenv	^{5}.1.0

Table C.1 – Packages Principaux Utilisés

Annexe D : Glossaire

Provider Pattern de gestion d'état dans Flutter utilisant le pattern Observer

Widget Composant de base de l'interface utilisateur dans Flutter

StatefulWidget Widget qui peut changer d'état au cours de son cycle de vie

StatelessWidget Widget qui ne change jamais d'état

BuildContext Contexte de construction d'un widget dans l'arbre des widgets

Navigator Système de navigation de Flutter pour gérer la pile d'écrans

Route Chemin de navigation vers un écran spécifique

Hot Reload Fonctionnalité de Flutter permettant de voir les modifications en temps réel

JSON Serialization Processus de conversion d'objets Dart en JSON et vice versa

Async/Await Mots-clés Dart pour la programmation asynchrone

Future Représentation d'une valeur qui sera disponible dans le futur

Stream Séquence asynchrone d'événements